



KTH Royal Institute of Technology

Omogen Heap

Simon Lindholm, Johan Sannemo, Mårten Wiman

2026-08-04

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

template.cpp	18 lines
<pre>#include <bits/stdc++.h> // #pragma GCC optimize("O3,unroll-loops,fast-math") // #pragma GCC target("avx2,sse4.2,fma") using namespace std; #define rep(i, a, b) for(int i = a; i < (b); ++i) #define all(x) begin(x), end(x) #define allr(x) x.rbegin(), x.rend() #define sz(x) (int)(x).size() typedef long long ll; typedef pair<int, int> pii; typedef vector<int> vi; int main() { ios::sync_with_stdio(0); cin.tie(0); cout.tie(0); cin.exceptions(cin.failbit); }</pre>	
.bashrc	3 lines
<pre>alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \ -fsanitize=undefined,address' xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =◇</pre>	
.vimrc	6 lines
<pre>set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul sy on im jk <esc> im kj <esc> no ; : " Select region and then type :Hash to hash your selection. " Useful for verifying that there aren't mistypes. ca Hash w !cpp -dD -P -fpreprocessed \ tr -d '[:space:]' \ \ md5sum \ cut -c-6</pre>	
hash.sh	3 lines
<pre># Hashes a file, ignoring all whitespace and comments. Use for # verifying that code was correctly typed.</pre>	

cpp -dD -P -fpreprocessed tr -d '[:space:]' md5sum cut -c-6	
troubleshoot.txt	52 lines
<pre>Pre-submit: Write a few simple test cases if sample is not enough. Are time limits close? If so, generate max cases. Is the memory usage fine? Could anything overflow? Make sure to submit the right file.</pre>	
Wrong answer:	
<pre>Print your solution! Print debug output, as well. Are you clearing all data structures between test cases? Can your algorithm handle the whole range of input? Read the full problem statement again. Do you handle all corner cases correctly? Have you understood the problem correctly? Any uninitialized variables? Any overflows? Confusing N and M, i and j, etc.? Are you sure your algorithm works? What special cases have you not thought of? Are you sure the STL functions you use work as you think? Add some assertions, maybe resubmit. Create some testcases to run your algorithm on. Go through the algorithm for a simple case. Go through this list again. Explain your algorithm to a teammate. Ask the teammate to look at your code. Go for a small walk, e.g. to the toilet. Is your output format correct? (including whitespace) Rewrite your solution from the start or let a teammate do it.</pre>	
Runtime error:	
<pre>Have you tested all corner cases locally? Any uninitialized variables? Are you reading or writing outside the range of any vector? Any assertions that might fail? Any possible division by 0? (mod 0 for example) Any possible infinite recursion? Invalidated pointers or iterators? Are you using too much memory? Debug with resubmits (e.g. remapped signals, see Various).</pre>	
Time limit exceeded:	
<pre>Do you have any possible infinite loops? What is the complexity of your algorithm? Are you copying a lot of unnecessary data? (References) How big is the input and output? (consider scanf) Avoid vector, map. (use arrays/unordered_map) What do your teammates think about your algorithm?</pre>	
Memory limit exceeded:	
<pre>What is the max amount of memory your algorithm should need? Are you clearing all data structures between test cases?</pre>	
Mathematics (2)	
2.0.1 Division Functions	
FloorCeilDivision.h	
Description: Mathematical floor and ceiling division. Standard C++ integer division truncates towards zero, which is incorrect for negative numbers. These functions correctly compute $\lfloor a/b \rfloor$ and $\lceil a/b \rceil$ for all signed integers using bitwise sign checking.	
Time: $\mathcal{O}(1)$	63f4f1, 7 lines
<pre>ll floor_div(ll a, ll b) {</pre>	

<pre> return a / b - ((a ^ b) < 0 && a % b != 0); }</pre>	
<pre>ll ceil_div(ll a, ll b) { return a / b + ((a ^ b) > 0 && a % b != 0); }</pre>	

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Any two complex numbers z_1 and z_2 are the roots of the quadratic equation:

$$z^2 - (z_1 + z_2)z + z_1z_2 = 0$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by = e &\Rightarrow x = \frac{ed - bf}{ad - bc} \\ cx + dy = f &\Rightarrow y = \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g.

$$a_n = (d_1n + d_2)r^n.$$

2.3 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths a, b, c ; semiperimeter $p = \frac{a+b+c}{2}$; area $A = \sqrt{p(p-a)(p-b)(p-c)}$.

Circumradius $R = \frac{abc}{4A}$; inradius $r = \frac{A}{p}$.

Centroid: $(x, y) = \frac{A+B+C}{3}$.

Median $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$; bisector $s_a = \sqrt{bc[1 - (\frac{a}{b+c})^2]}$.

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$.

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$.

Law of tangents: $\frac{a+b}{a-b} = \tan \frac{\alpha+\beta}{2} / \tan \frac{\alpha-\beta}{2}$.

2.4.2 Quadrilaterals

Side lengths a, b, c, d , diagonals e, f , angle θ , area A , flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals: $\sum \text{opp. angles} = 180^\circ$, $ef = ac + bd$, $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Common Shapes

Sphere: $V = \frac{4}{3}\pi r^3, S = 4\pi r^2$.

Cylinder: $V = \pi r^2 h, S = 2\pi r^2 + 2\pi r h$.

Cone: $V = \frac{1}{3}\pi r^2 h, S = \pi r^2 + \pi r \sqrt{r^2 + h^2}$.

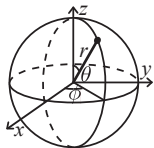
Pyramid: $V = \frac{1}{3}Bh$.

Torus: $V = 2\pi^2 R r^2, S = 4\pi^2 R r$ (R is dist. to center of tube).

Circle: $A = \pi r^2, C = 2\pi r$.

Ellipse: $A = \pi ab, C \approx \pi[3(a+b) - \sqrt{(3a+b)(a+3b)}]$.

2.4.4 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned} \frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1+x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) \end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sequences

2.6.1 Arithmetic Sequence

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= \frac{n(n+1)}{2} \\ 1^2 + 2^2 + 3^2 + \dots + n^2 &= \frac{n(2n+1)(n+1)}{6} \\ 1^3 + 2^3 + 3^3 + \dots + n^3 &= \frac{n^2(n+1)^2}{4} \\ 1^4 + 2^4 + 3^4 + \dots + n^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \end{aligned}$$

Sum of first n terms with difference d :

$$S_n = \frac{(n \cdot [2a_1 + (n-1)d])}{2}$$

Sum of first n triangular numbers:

$$S_n = \frac{n(n+1)(n+2)}{6}$$

Sum of first n odd numbers:

$$1 + 3 + 5 + \dots + (2n-1) = n^2$$

Sum of first n even numbers:

$$2 + 4 + 6 + \dots + 2n = n(n+1)$$

2.6.2 Geometric Sequence

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

n^{th} term of geometric sequence:

$$a_n = a_1 r^{n-1}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\text{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$
$$\mu = \lambda, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$
$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is

$$t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k.$$

2.10 Linear Algebra

XORBasis.h
Description: Linear basis for XOR. 'add' returns true if x increased the basis size. 'has' returns true if x can be formed by the basis.
Time: $\mathcal{O}(\textit{bits})$

fa25ec, 43 lines

```
struct Basis {
  ll b[62]{};
  int sz = 0;
  bool add(ll x) {
    for(int i=61; i>=0; i--) if(x >> i & 1)
      if(b[i]) x ^= b[i]; else return b[i] = x, sz++,
        true;
    return false;
  }
  bool has(ll x) {
    for(int i=61; i>=0; i--) if(x >> i & 1) x ^= b[i];
    return !x;
  }
  ll query(ll x = 0) {
    for(int i=61; i>=0; i--) x = max(x, x ^ b[i]);
    return x;
  }
  ll kth(ll k) {
    if (k < 1 || k > (1LL << sz)) return -1;
    ll x = 0, cnt = 1LL << sz;
    for (int i=61; i>=0; i--) if (b[i]) {
      cnt >>= 1;
      bool want = k > cnt;
      if (want) k -= cnt;
      if ((x >> i) & 1) != want) x ^= b[i];
    }
    return x;
  }
}

ll count_lt(ll x) {
  if (x < 0) return 0;
  ll ans = 0, mask = 0, cnt = 1LL << sz;
  for (int i=61; i>=0; i--) {
    if (b[i]) {
      cnt >>= 1;
      bool want = x >> i & 1;
      if (want) ans += cnt;
      if ((mask >> i) & 1) != want) mask ^= b[i];
    } else if (((x ^ mask) >> i) & 1) {
      return ans + ((x >> i & 1)? cnt : 0);
    }
  }
  return ans;
}
```

```
}
};

Data structures (3)

OrderStatisticTree.h
Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type.
Time:  $\mathcal{O}(\log N)$ 
<ext/pb_ds/assoc.container.hpp>, <ext/pb_ds/tree_policy.hpp> 60766f, 15 lines

using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;

void example() {
  ordered_set<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}

HashMap.h
Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).
d77092, 7 lines

#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = 11(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({}, {}, {}, {}, {1<<16});

IterativeSegmentTree.h
Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.
Time:  $\mathcal{O}(\log N)$ 
0f4bdb, 19 lines

struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

RecSegmentTree.h

Description: Recursive Segment Tree. Supports point update and range queries on [l, r] (inclusive). Generic implementation: change 'T', 'unit', and 'f' for different operations.
Time: $\mathcal{O}(N)$ build, $\mathcal{O}(\log N)$ update/query.

42ef0f, 40 lines

```
template <typename T>
struct SegTree {
    static constexpr T unit = 0;
    T f(T a, T b) { return a + b; } // (associative fn)
    vector<T> s; int n;
    SegTree(const vector<T>& v) : s(4*sz(v)), n(sz(v)) {
        build(0, n-1, 1, v);
    }
    void build(int l, int r, int u, const vector<T>& v) {
        if (l == r) s[u] = v[l];
        else {
            int mid = (l + r) / 2;
            build(l, mid, 2 * u, v);
            build(mid + 1, r, 2 * u + 1, v);
            s[u] = f(s[2 * u], s[2 * u + 1]);
        }
    }
    void update(int idx, T val, int l, int r, int u) {
        if (l == r) s[u] = val;
        else {
            int mid = (l + r) / 2;
            if (idx <= mid) update(idx, val, l, mid, 2 * u);
            else update(idx, val, mid + 1, r, 2 * u + 1);
            s[u] = f(s[2 * u], s[2 * u + 1]);
        }
    }
    T query(int ql, int qr, int l, int r, int u) {
        if (ql > r || qr < l) return unit;
        if (l >= ql && r <= qr) return s[u];
        int mid = (l + r) / 2;
        return f(query(ql, qr, l, mid, 2 * u),
            query(ql, qr, mid + 1, r, 2 * u + 1));
    }
    void update(int idx, T val) {
        update(idx, val, 0, n - 1, 1);
    }
    T query(int l, int r) {
        return query(l, r, 0, n - 1, 1);
    }
};
```

LazySegmentTree.h

Description: Segment Tree with Lazy Propagation.
Time: $\mathcal{O}(N)$ build, $\mathcal{O}(\log N)$ update/query.

25499f, 51 lines

```
template <typename T>
struct LazySegTree {
    int n;
    vector<T> tree;
    vector<int> lazy;
    LazySegTree(const vector<T> &a) : n(a.size()), tree(4 * n),
        lazy(4 * n) {
        build(1, 0, n - 1, a);
    }
    T f(T a, T b) { return a + b; } // (associative fn)
    void build(int v, int l, int r, const vector<T> &a) {
        if (l == r) {
            tree[v] = a[l];
        } else {
            int m = (l + r) / 2;
            build(2 * v, l, m, a);
            build(2 * v + 1, m + 1, r, a);
            tree[v] = f(tree[2 * v], tree[2 * v + 1]);
        }
    }
};
```

```
}
void apply(int v, int len, int x) {
    tree[v] += x*len;
    lazy[v] += x;
}

void push(int v, int l, int r) {
    int m = (l + r) / 2;
    apply(2 * v, m - l + 1, lazy[v]);
    apply(2 * v + 1, r - m, lazy[v]);
    lazy[v] = 0;
}

void update(int ql, int qr, int x, int v, int l, int r) {
    if (qr < l || ql > r) return;
    if (l >= ql && r <= qr) {
        apply(v, r - l + 1, x);
    } else {
        push(v, l, r);
        int m = (l + r) / 2;
        update(ql, qr, x, 2 * v, l, m);
        update(ql, qr, x, 2 * v + 1, m + 1, r);
        tree[v] = f(tree[2 * v], tree[2 * v + 1]);
    }
}

T query(int ql, int qr, int v, int l, int r) {
    if (qr < l || ql > r) return 0;
    if (l >= ql && r <= qr) return tree[v];
    push(v, l, r);
    int m = (l + r) / 2;
    return f(query(ql, qr, 2 * v, l, m), query(ql, qr, 2 *
        v + 1, m + 1, r));
}

void update(int ql, int qr, int x) { update(ql, qr, x, 1,
    0, n - 1); }

T query(int ql, int qr) { return query(ql, qr, 1, 0, n - 1)
    ; }
```

};

MaxSubarraySumSegmentTree.h

Description: Segment tree for maximum subarray sum. Each node stores max prefix sum, max suffix sum, max subarray sum, and total sum. All except total sum are ≥ 0 .
Time: $\mathcal{O}(\log N)$

dbbf39, 57 lines

```
struct Node {
    ll ans, suf, pref, sum;
    Node() : ans(0), suf(0), pref(0), sum(0) {}
    Node(ll val) {
        sum = val;
        ans = suf = pref = max(0LL, val);
    }
};

template <typename T>
struct SegTree {
    Node unit{};
    Node f(Node a, Node b) {
        Node ret;
        ret.sum = a.sum + b.sum;
        ret.pref = max(a.pref, a.sum + b.pref);
        ret.suf = max(b.suf, b.sum + a.suf);
        ret.ans = max({a.ans, b.ans, a.suf + b.pref});
        return ret;
    }
    vector<Node> s; int n;
    SegTree(const vector<T>& v) : s(4 * sz(v)), n(sz(v)) {
        if (n > 0) build(0, n - 1, 1, v);
    }
    void build(int l, int r, int u, const vector<T>& v) {
        if (l == r) s[u] = Node(v[l]);
    }
};
```

```
else {
    int mid = (l + r) / 2;
    build(l, mid, 2 * u, v);
    build(mid + 1, r, 2 * u + 1, v);
    s[u] = f(s[2 * u], s[2 * u + 1]);
}
}

void update(int idx, T val, int l, int r, int u) {
    if (l == r) s[u] = Node(val);
    else {
        int mid = (l + r) / 2;
        if (idx <= mid) update(idx, val, l, mid, 2 * u);
        else update(idx, val, mid + 1, r, 2 * u + 1);
        s[u] = f(s[2 * u], s[2 * u + 1]);
    }
}

Node query(int ql, int qr, int l, int r, int u) {
    if (ql > r || qr < l) return unit;
    if (l >= ql && r <= qr) return s[u];
    int mid = (l + r) / 2;
    return f(query(ql, qr, l, mid, 2 * u),
        query(ql, qr, mid + 1, r, 2 * u + 1));
}

void update(int idx, T val) {
    update(idx, val, 0, n - 1, 1);
}

ll query(int l, int r) {
    if (l > r) return 0;
    return query(l, r, 0, n - 1, 1).ans;
}
};
```

RangeGCD.h

Description: Segment tree for range GCD with range additions. Based on the property $\gcd(a_1, \dots, a_r) = \gcd(a_l, \gcd_{i=l}^r(a_i - a_l))$.
Time: $\mathcal{O}(\log N \cdot \log(\max A_i))$

d2dfd5, 62 lines

```
struct Node {
    ll first, g, lazy;
    bool has_lazy, empty;
    Node() : first(0), g(0), lazy(0), has_lazy(false), empty(true) {}
    Node(ll v) : first(v), g(0), lazy(0), has_lazy(false), empty(false) {}
    void apply(ll v) {
        if (!empty) first += v, lazy += v, has_lazy = true;
    }
};

struct RangeGCD {
    Node merge(const Node& a, const Node& b) {
        if (a.empty) return b;
        if (b.empty) return a;
        Node res; res.empty = false;
        res.first = a.first;
        res.g = __gcd(abs(a.g), __gcd(abs(b.g), abs(a.first - b.first)));
        return res;
    }
    int n;
    vector<Node> tree;
    RangeGCD(const vector<ll>& v) : n(sz(v)), tree(4 * n) {
        if (n > 0) build(1, 0, n - 1, v);
    }
    void build(int u, int l, int r, const vector<ll>& v) {
        if (l == r) { tree[u] = Node(v[l]); return; }
        int mid = (l + r) / 2;
        build(2 * u, l, mid, v);
        build(2 * u + 1, mid + 1, r, v);
    }
};
```

```
        tree[u] = merge(tree[2 * u], tree[2 * u + 1]);
    }
    void push(int u) {
        if (tree[u].has_lazy) {
            tree[2 * u].apply(tree[u].lazy);
            tree[2 * u + 1].apply(tree[u].lazy);
            tree[u].lazy = 0; tree[u].has_lazy = false;
        }
    }
    void update(int ql, int qr, ll v, int u, int l, int r) {
        if (ql <= l && r <= qr) { tree[u].apply(v); return; }
        push(u);
        int mid = (l + r) / 2;
        if (ql <= mid) update(ql, qr, v, 2 * u, l, mid);
        if (qr > mid) update(ql, qr, v, 2 * u + 1, mid + 1, r);
        tree[u] = merge(tree[2 * u], tree[2 * u + 1]);
    }
    Node query(int ql, int qr, int u, int l, int r) {
        if (ql <= l && r <= qr) return tree[u];
        push(u);
        int mid = (l + r) / 2;
        if (qr <= mid) return query(ql, qr, 2 * u, l, mid);
        if (ql > mid) return query(ql, qr, 2 * u + 1, mid + 1, r);
        return merge(query(ql, qr, 2 * u, l, mid),
            query(ql, qr, 2 * u + 1, mid + 1, r));
    }
    void add(int l, int r, ll v) { if (l <= r) update(l, r, v, 1,
        0, n - 1); }
    ll query(int l, int r) {
        if (l > r) return 0;
        Node res = query(l, r, 1, 0, n - 1);
        return __gcd(abs(res.first), abs(res.g));
    }
};
```

DynamicSegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory. It handles huge coordinates (e.g., indices up to 10^9 or 10^{18}) without coordinate compression.

Usage: Node* tr = new Node(v, 0, sz(v));

Time: $\mathcal{O}(\log N)$.

```
"/various/BumpAllocator.h" 34ecf5, 50 lines

const int inf = 1e9;
struct Node {
    Node *l = 0, *r = 0;
    int lo, hi, mset = inf, madd = 0, val = -inf;
    Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
    Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
        if (lo + 1 < hi) {
            int mid = lo + (hi - lo)/2;
            l = new Node(v, lo, mid); r = new Node(v, mid, hi);
            val = max(l->val, r->val);
        }
        else val = v[lo];
    }
    int query(int L, int R) { // range [L, R)
        if (R <= lo || hi <= L) return -inf;
        if (L <= lo && hi <= R) return val;
        push();
        return max(l->query(L, R), r->query(L, R));
    }
    void set(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) mset = val = x, madd = 0;
        else {
            push(), l->set(L, R, x), r->set(L, R, x);
            val = max(l->val, r->val);
        }
    }
    void add(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) mset = val = x, madd = 0;
        else {
            push(), l->add(L, R, x), r->add(L, R, x);
            val = max(l->val, r->val);
        }
    }
};
```

```
    }
    void add(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            if (mset != inf) mset += x;
            else madd += x;
            val += x;
        }
        else {
            push(), l->add(L, R, x), r->add(L, R, x);
            val = max(l->val, r->val);
        }
    }
    void push() {
        if (!l) {
            int mid = lo + (hi - lo)/2;
            l = new Node(lo, mid); r = new Node(mid, hi);
        }
        if (mset != inf)
            l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
        else if (madd)
            l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
    }
};
```

BetterDynamicSegmentTree.h

Description: Pointer-based dynamic segment tree with lazy propagation (range set). Can use bump allocator for speed. Only creates nodes when necessary.

Usage: Tree* root = new Tree(0, 1e9);

Time: $\mathcal{O}(\log N)$ per update/query.

```
"/various/BumpAllocator.h" 1df32b, 35 lines

struct Tree {
    Tree *l = 0, *r = 0;
    int lo, hi;
    ll sum = 0, lz = 0;
    bool has = 0; // is_lazy
    Tree(int L, int R) : lo(L), hi(R) {}

    void apply(ll v) { sum = (ll)(hi - lo) * v; lz = v; has = 1; }

    void push() {
        if (lo + 1 == hi || !has) return;
        int m = lo + (hi - lo) / 2;
        if (!l) l = new Tree(lo, m); if (!r) r = new Tree(m, hi);
        l->apply(lz); r->apply(lz);
        has = 0;
    }

    void update(int L, int R, ll v) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) return apply(v);
        push();
        int m = lo + (hi - lo) / 2;
        if (L < m) { if (!l) l = new Tree(lo, m); l->update(L, R, v); }
        if (R > m) { if (!r) r = new Tree(m, hi); r->update(L, R, v); }
        sum = (l ? l->sum : 0) + (r ? r->sum : 0);
    }

    ll query(int L, int R) {
        if (R <= lo || hi <= L) return 0;
        if (L <= lo && hi <= R) return sum;
        if (has) return (ll)(min(R, hi) - max(L, lo)) * lz;
        int m = lo + (hi - lo) / 2;
    }
};
```

```
        return (l && L < m ? l->query(L, R) : 0) + (r && R > m
            ? r->query(L, R) : 0);
    }
};

SparseSegmentTree.h
Description: Dynamic/sparse segment tree using an array/vector-based representation. Supports range set updates and range sum queries.
Usage: SparseSegmentTree<> tree(N);
Time:  $\mathcal{O}(\log N)$  per update/query.
513978, 69 lines

struct Node {
    ll val = 0, lazy = 0;
    bool is_lazy = false;
    int left = -1, right = -1;

    void apply(ll x, int l, int r) {
        val = x*(r-l+1);
        lazy = x;
        is_lazy = true;
    }
};

template<typename T = ll>
struct SparseSegmentTree {
#define L tree[node].left
#define R tree[node].right
#define MID ((l+r)>>1)
private:
    int sz, timer=0;
    vector<Node> tree;
    T identity;

    inline void grow(int &node) {
        if (L == -1) { L = ++timer; tree.push_back(Node()); }
        if (R == -1) { R = ++timer; tree.push_back(Node()); }
    }

    void propagate(int node, int l, int r){
        if(!tree[node].is_lazy || l == r) return;
        tree[L].apply(tree[node].lazy, l, MID);
        tree[R].apply(tree[node].lazy, MID+1, r);
        tree[node].is_lazy = false; tree[node].lazy = 0;
    }

    void update(int node, int l, int r, int ql, int qr, T val)
    {
        if (l > qr || r < ql) return;
        if (ql <= l && r <= qr) {
            tree[node].apply(val, l, r);
            return;
        }
        grow(node);
        propagate(node, l, r);
        update(L, l, MID, ql, qr, val);
        update(R, MID + 1, r, ql, qr, val);
        tree[node].val = tree[L].val + tree[R].val;
    }

    T query(int node, int l, int r, int ql, int qr) {
        if (l > qr || r < ql) return identity;
        if (ql <= l && r <= qr) return tree[node].val;
        grow(node);
        propagate(node, l, r);
        ll res = 0;
        res += query(L, l, MID, ql, qr);
        res += query(R, MID + 1, r, ql, qr);
        return res;
    }
public:
```

```

SparseSegmentTree(int n, int q=0) : sz(n) {
    if (q > 0) { tree.reserve(100 * q); }
    tree.push_back(Node()); // root node
    identity=0;
}
void update(int lq, int rq, T val) { update(0, 0, sz-1, lq,
    rq, val); }
T query(int lq, int rq) { return query(0, 0, sz-1, lq, rq);
}

#undef L
#undef R
#undef MID
};

```

Sparse2D.h

Description: 2D Sparse Table for idempotent operations (GCD, min, max) on rectangular 0-indexed subgrids.

Usage: Sparse2D<ll> st(grid);

ll ans = st.query(r1, c1, r2, c2); // inclusive bounds [r1..r2] x [c1..c2]

Time: $\mathcal{O}(N * M * \log N * \log M)$ build, $\mathcal{O}(1)$ query.

287317, 48 lines

```

template<class T = long long>
struct Sparse2D { // 0-indexed, rectangular subgrid queries
    int n, m;
    vector<int> lg2;
    vector<vector<vector<vector<T>>>> st;

    T combine(T a, T b) {return gcd(a, b); }

    Sparse2D(const vector<vector<T>> &z) : n(sz(z)), m(sz(z[0])) {
        int lim = max(n, m);
        lg2.resize(lim + 1);
        for (int i = 2; i <= lim; i++) lg2[i] = lg2[i >> 1] + 1;

        int LG1 = lg2[n] + 1;
        int LG2 = lg2[m] + 1;
        st.assign(n, vector<vector<vector<T>>>(m, vector<vector<T>>>(LG1, vector<T>(LG2))));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                st[i][j][0][0] = z[i][j];
            }
        }

        for (int a = 0; a < LG1; a++) {
            for (int b = 0; b < LG2; b++) {
                if (a + b == 0) continue;
                for (int i = 0; i + (1 << a) <= n; i++) {
                    for (int j = 0; j + (1 << b) <= m; j++) {
                        if (!a) {
                            st[i][j][a][b] = combine(st[i][j][a]
                                ][b - 1], st[i][j] + (1 << (b - 1))][a][b - 1]);
                        } else {
                            st[i][j][a][b] = combine(st[i][j][a]
                                - 1][b], st[i + (1 << (a - 1))][j][a - 1][b]);
                        }
                    }
                }
            }
        }
    }

    T query(int x1, int y1, int x2, int y2) {

```

```

        x2++; y2++;
        int a = lg2[x2 - x1], b = lg2[y2 - y1];
        return combine(
            combine(st[x1][y1][a][b], st[x2 - (1 << a)][y1][a][b]),
            combine(st[x1][y2 - (1 << b)][a][b], st[x2 - (1 << a)][y2 - (1 << b)][a][b])
        );
    }
};

```

MergeSortTree.h

Description: Segment tree where each node holds a sorted vector of the range. Useful for querying the number of elements $> V$ in range $[L, R]$. Replace `upper_bound()` with `lower_bound()` to query $\geq v$.

Memory: $\mathcal{O}(N \log N)$

Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(\log^2 N)$ query.

d65335, 30 lines

```

template <typename T>
struct MergeSortTree {
    int n;
    vector<vector<T>> tree;
    MergeSortTree(vector<T>& a) : n(sz(a)), tree(4*n) {
        build(1, 0, n-1, a);
    }
    void build(int node, int l, int r, vector<T>& a) {
        if (l == r) {
            tree[node] = {a[l]};
            return;
        }
        int mid = (l + r) / 2;
        build(2*node, l, mid, a);
        build(2*node+1, mid+1, r, a);
        merge(all(tree[2*node]), all(tree[2*node+1]),
            back_inserter(tree[node]));
    }
    T query(int node, int tl, int tr, int l, int r, T v) {
        if (l > tr || r < tl) return 0;
        if (l <= tl && tr <= r) {
            return tree[node].end() - upper_bound(all(tree[node]
                ), v);
        }
        int mid = (tl + tr) / 2;
        return query(2*node, tl, mid, l, r, v) +
            query(2*node+1, mid+1, tr, l, r, v);
    }
    T query(int l, int r, T v) { // count of numbers in a[l..r]
        greater than v
        return query(1, 0, n-1, l, r, v);
    }
};

```

PersistentSegTree.h

Description: persistent segment tree used to track previous changes and rollback to a certain point in time. verified using: <https://cses.fi/problemset/task/1737>

cbfb6b, 53 lines

```

#define int ll

struct pSeg{
    struct node{
        ll sum = 0;
        node *l , *r;
        node(ll val): sum(val), l(nullptr), r(nullptr){
            node(node *l , node *r): l(l), r(r){
                sum = l->sum + r->sum;
            }
        }
        node(node * cp): sum(cp->sum), l(cp->l), r(cp->r){
    };
};

```

```

vector<node*> root;
int sz;

private:
node* build(int l, int r, vector<ll> &a){
    if(l == r){
        return new node(a[l]);
    }
    int m = (l+r)/2;
    return new node(build(l,m,a) , build(m+1 , r, a));
}
node* set(node *v , int l, int r, int pos, ll val){
    if(l == r){
        return new node(val);
    }
    int m = (l + r)/2;
    if(pos <= m) return new node(set(v->l , l , m , pos ,
        val), v->r);
    else return new node(v->l , set(v->r, m + 1, r , pos,
        val));
}
ll range_sum(node *v, int l, int r, int ll, int rl){
    if(rl < l || ll > r)
        return 0;
    if(r <= rl && l >= ll)
        return v->sum;
    int m = (l+r)/2;
    return range_sum(v->l, l, m, ll, rl) + range_sum(v->r ,
        m + 1, r, ll, rl);
}

public:
pSeg(vector<ll> arr): sz(arr.size()){
    root.push_back(build(0 , sz - 1,arr));
}
void set(int idx, int pos, ll val){
    root[idx] = set(root[idx] , 0, sz-1, pos, val);
}
ll range_sum(int idx, int l, int r){
    return range_sum(root[idx] , 0 , sz-1, l , r);
}
void copy(int idx){
    root.push_back(new node(root[idx]));
}
};

```

PersistentSegTreeArrayBased.h

Description: array-based persistent segment tree used to track previous changes and rollback to a certain point in time.

1d3c4d, 68 lines

```

struct pSeg {
    struct Node {
        ll sum;
        int l, r;
    };

    vector<Node> tree;
    vector<int> root;
    int sz;

    pSeg(vector<ll>& arr) : sz(arr.size()) {
        tree.reserve(6000000); // Pre-allocate to prevent
            reallocation MLE
        tree.push_back({0, 0, 0}); // Dummy node at index 0
        root.push_back(build(0, sz - 1, arr));
    }

    int new_node(ll sum, int l = 0, int r = 0) {
        tree.push_back({sum, l, r});
        return tree.size() - 1;
    }
};

```

```
    }

    int build(int l, int r, vector<ll>& a) {
        if (l == r) return new_node(a[l]);
        int m = (l + r) / 2;
        int left_child = build(l, m, a);
        int right_child = build(m + 1, r, a);
        return new_node(tree[left_child].sum ^ tree[right_child]
            ].sum, left_child, right_child);
    }

    int set(int v, int l, int r, int pos, ll val) {
        if (l == r) return new_node(val);
        int m = (l + r) / 2;
        if (pos <= m) {
            int left_child = set(tree[v].l, l, m, pos, val);
            return new_node(tree[left_child].sum ^ tree[tree[v]
                ].r].sum, left_child, tree[v].r);
        } else {
            int right_child = set(tree[v].r, m + 1, r, pos, val
                );
            return new_node(tree[tree[v].l].sum ^ tree[
                right_child].sum, tree[v].l, right_child);
        }
    }

    void copy(int idx) {
        root.push_back(root[idx]);
    }

    void set(int idx, int pos, ll val) {
        root[idx] = set(root[idx], 0, sz - 1, pos, val);
    }

    ll query(int node_l, int node_r, int l, int r) {
        ll res = tree[node_l].sum ^ tree[node_r].sum;
        if (res == 0) return -1;
        if (l == r) return l;

        int m = (l + r) / 2;
        ll left_res = tree[tree[node_l].l].sum ^ tree[tree[
            node_r].l].sum;

        if (left_res > 0) {
            return query(tree[node_l].l, tree[node_r].l, l, m);
        } else {
            return query(tree[node_l].r, tree[node_r].r, m + 1,
                r);
        }
    }

    ll query(ll l, ll r) {
        return query(root[l - 1], root[r], 0, sz - 1);
    }
};
```

ImplicitPersistentSegTree.h

Description: Implicit Persistent Segment Tree. Operates on a dynamic value range [MIN_VAL, MAX_VAL] without coordinate compression. reserve_nodes = $N \times (\lceil \log_2(\text{MAX_VAL} - \text{MIN_VAL}) \rceil + 2)$
Memory: $\mathcal{O}(N \log(\text{MAX} - \text{MIN}))$
Time: $\mathcal{O}(\log(\text{MAX} - \text{MIN}))$ per query/update.

e66fcf, 55 lines

```
struct PSegTree {
    struct Node {
        ll val = 0;
        int l = 0, r = 0;
    };
};
```

```
vector<Node> tree;
vector<int> roots;
ll MIN_VAL, MAX_VAL;

PSegTree(ll min_v, ll max_v, int reserve_nodes = 0)
    : MIN_VAL(min_v), MAX_VAL(max_v) {
    if (reserve_nodes > 0) tree.reserve((size_t)reserve_nodes);
    tree.push_back({}); // Index 0 is the universal null/empty
        node
    roots.push_back(0);
}

int update(int node, ll L, ll R, ll pos, ll val) {
    int cur = (int)tree.size();
    tree.push_back(tree[(size_t)node]);
    if (L == R) {
        tree[(size_t)cur].val += val; // Change to ‘=’ if
            overwriting is preferred
        return cur;
    }
    ll mid = L + (R - L) / 2;
    if (pos <= mid) {
        tree[(size_t)cur].l = update(tree[(size_t)node].l, L, mid
            , pos, val);
    } else {
        tree[(size_t)cur].r = update(tree[(size_t)node].r, mid +
            1, R, pos, val);
    }
    tree[(size_t)cur].val = tree[(size_t)tree[(size_t)cur].l].
        val + tree[(size_t)tree[(size_t)cur].r].val;
    return cur;
}

// Creates a new version of the tree based on an existing
    root
int update(int root_idx, ll pos, ll val) {
    int new_root = update(roots[(size_t)root_idx], MIN_VAL,
        MAX_VAL, pos, val);
    roots.push_back(new_root);
    return new_root;
}

ll query(int node, ll L, ll R, ll ql, ll qr) {
    if (!node || L > qr || R < ql) return 0;
    if (ql <= L && R <= qr) return tree[(size_t)node].val;

    ll mid = L + (R - L) / 2;
    return query(tree[(size_t)node].l, L, mid, ql, qr) +
        query(tree[(size_t)node].r, mid + 1, R, ql, qr);
}

// Queries a specific version of the tree
ll query(int root_idx, ll ql, ll qr) {
    return query(roots[(size_t)root_idx], MIN_VAL, MAX_VAL, ql,
        qr);
}
};
```

ImplicitPersistentSegTreeArrayBased.h

Description: Array-based Implicit Persistent Segment Tree. Operates on a dynamic value range without coordinate compression.
Memory: $\mathcal{O}(N \log(\text{domain_size}))$
Time: $\mathcal{O}(\log(\text{domain_size}))$ per query/update.

d7f95f, 33 lines

```
const int N = 2e5 + 5;
const int MAX_NODES = N * (__lg(1000000001) + 2); // N * (-lg(
    domain_size) + 2)
int lc_arr[MAX_NODES];
```

```
int rc_arr[MAX_NODES];
ll val_arr[MAX_NODES];
int roots[N]; // Assuming n <= 200000
int node_cnt = 0;

int update(int node, ll L, ll R, ll pos, ll val) {
    int cur = ++node_cnt;
    lc_arr[cur] = lc_arr[node];
    rc_arr[cur] = rc_arr[node];
    val_arr[cur] = val_arr[node] + val;

    if (L == R) return cur;

    ll mid = L + (R - L) / 2;
    if (pos <= mid) {
        lc_arr[cur] = update(lc_arr[node], L, mid, pos, val);
    } else {
        rc_arr[cur] = update(rc_arr[node], mid + 1, R, pos, val);
    }
    return cur;
}

ll query(int node, ll L, ll R, ll ql, ll qr) {
    if (!node || L > qr || R < ql) return 0;
    if (ql <= L && R <= qr) return val_arr[node];

    ll mid = L + (R - L) / 2;
    return query(lc_arr[node], L, mid, ql, qr) +
        query(rc_arr[node], mid + 1, R, ql, qr);
}
```

UnionFind.h

Description: Disjoint-set data structure with path compression.
Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```
struct UF {
    vi e;
    UF(int n) : e(n, -1) {}
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        e[a] += e[b]; e[b] = a;
        return true;
    }
};
```

WeightedDSU.h

Description: Disjoint-set data structure with path compression. Maintains relative weights/distances between nodes in the same component. Union(u, v, w) adds a directed relationship representing $\text{Value}(u) - \text{Value}(v) = w$. Returns false if the new relationship contradicts existing paths (cycle detection).
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(\alpha(N))$ amortized per operation.

8fb576, 26 lines

```
struct WeightedDSU {
    vi leader, sz;
    vector<ll> weight;

    WeightedDSU(int n) : leader(n+1, -1), sz(n+1, 1), weight(n
        +1) {}
    int find(int u) {
        if(leader[u]<0) return u;
        int root = find(leader[u]); // Recursion first!
        weight[u] += weight[leader[u]];
    }
};
```

```
        return leader[u] = root;
    }

    bool Union(int u, int v, long long w) {
        int root_u = find(u);
        int root_v = find(v);
        if (root_u != root_v) {
            leader[root_u] = root_v;
            sz[root_v] += sz[root_u];
            weight[root_u] = w - weight[u] + weight[v]; //
                Apply the derived formula
            return true;
        } else {
            // Check consistency: (u - root) - (v - root) == w
            return (weight[u] - weight[v] == w);
        }
    }
};
```

BipartiteDSU.h

Description: Disjoint-set data structure with path compression and union by size. Maintains the parity of the path length from each node to its component's root. Can be used to check if a dynamically growing graph remains bipartite. The global is_bipartite flag permanently becomes false if an odd cycle is added.
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(\alpha(N))$ amortized per operation.

aalc74, 24 lines

```
struct BipartiteDSU {
    vi e, parity;
    bool is_bipartite = true;
    BipartiteDSU(int n) : e(n, -1), parity(n, 0) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) {
        if (e[x] < 0) return x;
        int root = find(e[x]);
        parity[x] ^= parity[e[x]];
        return e[x] = root;
    }
    void add_edge(int aa, int bb) {
        int a = find(aa), b = find(bb);
        if (a == b) {
            if (parity[aa] == parity[bb]) {
                is_bipartite = false;
            }
        } else {
            if (e[a] > e[b]) swap(a, b);
            e[a] += e[b]; e[b] = a;
            parity[b] = parity[aa] ^ parity[bb] ^ 1;
        }
    }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: int t = uf.time(); ...; uf.rollback(t);
Time: $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
struct RollbackUF {
    vi e; vector<pii> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i --> t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
};
```

```
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};
```

RollbackWeightedDSU.h

Description: Weighted Disjoint-set data structure with rollbacks. Maintains relative weights representing Value(u) - Value(v) = w. Uses a state stack to support strictly $\mathcal{O}(1)$ rollbacks. Uses Union-by-Size and NO path compression to preserve history. If a cycle is formed, join() pushes a dummy state and returns whether the weight is consistent.
Memory: $\mathcal{O}(N)$ for nodes, $\mathcal{O}(Q)$ for history stack.
Time: $\mathcal{O}(\log N)$ per find/join, $\mathcal{O}(1)$ per rollback step.

268c0d, 75 lines

```
struct RollbackWeightedDSU {
    vector<int> leader, sz;
    vector<long long> weight;
    int comps;

    // Struct to snapshot the exact state of the child node
    // before it was merged
    struct State {
        int u; // The node that became a child (-1 if dummy)
        int v; // The node that became the parent
        int sz_v; // Old size of the parent
        long long weight_u; // Old weight of u (usually 0)
    };
    vector<State> st;

    RollbackWeightedDSU(int n) : leader(n + 1), sz(n + 1, 1),
        weight(n + 1, 0), comps(n) {
        // Initialize 1-based indexing (or 0-based, works for
        // both)
        for (int i = 0; i <= n; i++) leader[i] = i;
    }

    // Returns {root, Value(x) - Value(root)}
    // Strictly O(log N) - NO path compression!
    pair<int, long long> find(int x) {
        if (x == leader[x]) return {x, 0};
        auto p = find(leader[x]);
        return {p.first, p.second + weight[x]}; // Accumulate
            distance to root
    }

    int time() { return st.size(); }

    void rollback(int t) {
        while (st.size() > t) {
            auto s = st.back();
            st.pop_back();

            if (s.u != -1) { // If an actual merge occurred
                leader[s.u] = s.u; // Restore u to being
                    its own root
                sz[s.v] = s.sz_v; // Restore parent's
                    size
                weight[s.u] = s.weight_u; // Restore u's weight
                comps++; // A component split
                    back into two
            }
        }
    }
};
```

```
    }

    // Attempts to add the relationship Value(u) - Value(v) = w
    bool join(int u, int v, long long w) {
        auto pu = find(u), pv = find(v);
        int root_u = pu.first, root_v = pv.first;
        long long dist_u = pu.second, dist_v = pv.second;

        if (root_u == root_v) {
            // Push a dummy state to keep time() perfectly
            // synchronized
            st.push_back({-1, -1, -1, 0});
            // Return true if the new edge is consistent, false
            // if it contradicts
            return (dist_u - dist_v == w);
        }

        // Union by size to guarantee O(log N) height
        if (sz[root_u] > sz[root_v]) {
            swap(root_u, root_v);
            swap(dist_u, dist_v);
            w = -w; // CRITICAL: If we swap the merge direction
                , the relative distance flips!
        }

        // Snapshot the state before modifying
        st.push_back({root_u, root_v, sz[root_v], weight[root_u]
            });

        leader[root_u] = root_v;
        sz[root_v] += sz[root_u];
        // The derived triangle math formula
        weight[root_u] = w - dist_u + dist_v;
        comps--;

        return true;
    }
};
```

RollbackBipartiteDSU.h

Description: Bipartite Disjoint-set data structure with rollbacks. Maintains the parity of the path length to the root to check for odd-length cycles. Uses a state stack to support strictly $\mathcal{O}(1)$ rollbacks to any previous time state. Strictly uses Union-by-Size and NO path compression to preserve tree topology.
Memory: $\mathcal{O}(N)$ for nodes, $\mathcal{O}(Q)$ for history stack.
Time: $\mathcal{O}(\log N)$ per operation, $\mathcal{O}(1)$ per rollback step.

9784bf, 70 lines

```
struct RollbackBipartiteUF {
    vi e, parity;
    int comps;
    int odd_cycles; // The graph is bipartite if odd_cycles ==
        0

    struct State {
        int u, e_u;
        int v, e_v, parity_v;
        int comps, odd_cycles;
    };
    vector<State> st;

    RollbackBipartiteUF(int n) : e(n, -1), parity(n, 0), comps(
        n), odd_cycles(0) {}
    int size(int x) { return -e[find(x).first]; }
    int time() { return sz(st); }
    bool is_bipartite() { return odd_cycles == 0; }

    // Returns {root, parity distance to root}
```

```
// O(log n) because we CANNOT use path compression with
// rollbacks
pii find(int x) {
    if (e[x] < 0) return {x, 0};
    pii p = find(e[x]);
    return {p.first, p.second ^ parity[x]};
}

void rollback(int t) {
    while (sz(st) > t) {
        auto s = st.back();
        st.pop_back();

        // If u != -1, an actual tree merge happened, so we
        // restore the arrays
        if (s.u != -1) {
            e[s.u] = s.e_u;
            e[s.v] = s.e_v;
            parity[s.v] = s.parity_v;
        }

        // Always restore the global counters
        comps = s.comps;
        odd_cycles = s.odd_cycles;
    }
}

bool join(int x, int y) {
    pii px = find(x), py = find(y);
    int u = px.first, v = py.first;
    int parity_x = px.second, parity_y = py.second;

    if (u == v) {
        // Same component! Check if they have the same
        // color.
        bool is_odd_cycle = (parity_x == parity_y);

        // Push a "dummy" state where u = -1 to keep track
        // of time()
        st.push_back({-1, -1, -1, -1, -1, comps, odd_cycles});

        if (is_odd_cycle) odd_cycles++;
        return false;
    }
    if (e[u] > e[v]) {
        swap(u, v);
        swap(parity_x, parity_y);
    }
    st.push_back({u, e[u], v, e[v], parity[v], comps,
        odd_cycles});
    e[u] += e[v];
    e[v] = u;
    parity[v] = parity_x ^ parity_y ^ 1;
    comps--;
    return true;
}

};
```

ParallelDSU.h
Description: Disjoint Set Union on ranges (Parallel DSU). Allows forcing two ranges [l1, r1] and [l2, r2] to be identical. Constraints are lazily propagated down from larger power-of-2 segments to smaller ones. After all unions, call propagate() to push everything to level 0. find(i, 0) gives the component ID of index i.
Time: Construction $\mathcal{O}(N \log N)$, Union $\mathcal{O}(\alpha(N))$, Propagation $\mathcal{O}(N \log N \alpha(N))$.

```
int n;
vector<vi> par;
RangeDSU(int n) : n(n) {
    int k = __lg(n) + 1;
    par.assign(k, vi(n));
    for (int i = 0; i < k; i++) iota(all(par[i]), 0);
}

int find(int i, int k) { // lvl k
    return par[k][i] == i ? i : par[k][i] = find(par[k][i], k);
}

void join(int i, int j, int k) {
    i = find(i, k); j = find(j, k);
    if (i != j) par[k][i] = j;
}

// Force range s[l1...r1] == s[l2...r2]
void join_range(int l1, int r1, int l2, int r2) {
    int len = r1 - l1 + 1;
    int k = __lg(len);
    join(l1, l2, k);
    join(r1 - (1 << k) + 1, r2 - (1 << k) + 1, k);
}

void propagate() {
    for (int i = par.size() - 1; i > 0; i--) {
        for (int j = 0; j <= n - (1 << i); j++) {
            int root = find(j, i);
            if (j != root) {
                join(j, root, i - 1);
                join(j + (1 << (i - 1)), root + (1 << (i - 1)), i - 1);
            }
        }
    }
}

};
```

SmallToLarge.h
Description: "Sack" or DSU on Trees. Solves offline subtree queries. 'val[u]' stores node values. 'dfs' populates 'ans'. The heavy child is lazily kept; light children are brute-forced. Fills 'add' with problem specific logic.
Time: $\mathcal{O}(N \log N)$

```
e13663, 46 lines

struct Sack {
    int n;
    vector<vi> adj;
    vi size, st, en, nodeAt;
    int timer;
    // Problem specific data
    vi val;
    Sack(int n) : n(n), adj(n), size(n), st(n), en(n), nodeAt(n), val(n), timer(0) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void prepare(int u, int p) {
        size[u] = 1;
        st[u] = timer; nodeAt[timer++] = u;
        for (auto &v : adj[u]) if (v != p) {
            prepare(v, u);
            size[u] += size[v];
            // Swap heavy child to front
            if (size[v] > size[adj[u][0]] || adj[u][0] == p)
                swap(v, adj[u][0]);
        }
        en[u] = timer;
    }
    void update(int u, int dir) {
        // dir = 1 (add), -1 (remove)
```

```
}

void dfs(int u, int p, bool keep) {
    int big = (adj[u].size() && adj[u][0] != p) ? adj[u][0] : -1;
    for (int v : adj[u])
        if (v != p && v != big) dfs(v, u, 0);
    if (big != -1) dfs(big, u, 1);
    for (auto v : adj[u]) {
        if (v != p && v != big) {
            rep(t, st[v], en[v]) update(nodeAt[t], 1);
        }
    }
    update(u, 1);

    // Answer query for u
    // ans[u] = ...

    if (!keep) {
        rep(t, st[u], en[u]) update(nodeAt[t], -1);
    }
}

};
```

SubMatrix.h
Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // top left 4 elements
Time: $\mathcal{O}(N^2 + Q)$

```
template<class T>
struct SubMatrix {
    vector<vector<T>>> p;
    SubMatrix(vector<vector<T>>& v) {
        int R = sz(v), C = sz(v[0]);
        p.assign(R+1, vector<T>(C+1));
        rep(r,0,R) rep(c,0,C)
            p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
    }
    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};
```

Matrix.h
Description: Basic operations on matrices.
Usage: Matrix<int, 3> A; // Square 3x3
Matrix<int, 3, 2> B; // Rectangular 3x2
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;

```
template<class T, int N, int M = N> struct Matrix {
    array<array<T, M>, N> d{};
    template<int P>
    Matrix<T, N, P> operator*(const Matrix<T, M, P>& m) const {
        Matrix<T, N, P> a;
        rep(i,0,N) rep(j,0,P) {
            a.d[i][j] += d[i][k] * m.d[k][j] % MOD;
            if(a.d[i][j]>=MOD) a.d[i][j] -= MOD;
        }
        return a;
    }
    array<T, N> operator*(const array<T, M>& vec) const {
        array<T, N> ret{};
        rep(i,0,N) rep(j,0,M) {
            ret[i] += d[i][j] * vec[j] % MOD;
            if(ret[i]>=MOD) ret[i] -= MOD;
        }
    }
};
```

```

    return ret;
}
Matrix operator^(ll p) const {
    static_assert(N == M, "Exponentiation requires square matrix");
    assert(p >= 0);
    Matrix a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
        if (p&1) a = a*b;
        b = b*b;
        p >>= 1;
    }
    return a;
}
};

```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”). If you want to find the minimum of $y = kx + m$, it is mathematically identical to finding the maximum of $y = (-k)x + (-m)$, and then negating the result.
Time: $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) { // the intersection of
        two adjacent lines x and y
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x->p >= y->p)
            isect(x, erase(y)));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

```

LiChaoTree.h

Description: Li Chao Tree for line and segment minimum queries. Supports add_line(m, c), add_segment(m, c, l, r), and query(x). For maximum, insert negated lines -m, -c and negate the query result.
Time: $\mathcal{O}(\log N)$

d049eb, 71 lines

```

const ll inf = 2e18;

struct Line {
    ll m, c;
    ll eval(ll x) { return m * x + c; }
};

```

LineContainer LiChaoTree PersistentLiChaoTree

```

struct node {
    Line line;
    node *left = nullptr, *right = nullptr;
    node(Line line) : line(line) {}
    void add_segment(Line nw, int l, int r, int L, int R) {
        if (l > r || r < L || l > R) return;
        int m = (l + l == r ? l : (l + r) / 2);
        if (l >= L && r <= R) {
            bool lef = nw.eval(l) < line.eval(l);
            bool mid = nw.eval(m) < line.eval(m);
            if (mid) swap(line, nw);
            if (l == r) return;
            if (lef != mid) {
                if (left == nullptr) left = new node(nw);
                else left->add_segment(nw, l, m, L, R);
            } else {
                if (right == nullptr) right = new node(nw);
                else right->add_segment(nw, m + 1, r, L, R);
            }
            return;
        }
        if (max(l, L) <= min(m, R)) {
            if (left == nullptr) left = new node({0, inf});
            left->add_segment(nw, l, m, L, R);
        }
        if (max(m + 1, L) <= min(r, R)) {
            if (right == nullptr) right = new node({0, inf});
            right->add_segment(nw, m + 1, r, L, R);
        }
    }
    ll query_segment(ll x, int l, int r, int L, int R) {
        if (l > r || r < L || l > R) return inf;
        int m = (l + l == r ? l : (l + r) / 2);
        if (l >= L && r <= R) {
            ll ans = line.eval(x);
            if (l < r) {
                if (x <= m && left != nullptr) ans = min(ans, left->
                    query_segment(x, l, m, L, R));
                if (x > m && right != nullptr) ans = min(ans, right->
                    query_segment(x, m + 1, r, L, R));
            }
            return ans;
        }
        ll ans = inf;
        if (max(l, L) <= min(m, R)) {
            if (left == nullptr) left = new node({0, inf});
            ans = min(ans, left->query_segment(x, l, m, L, R));
        }
        if (max(m + 1, L) <= min(r, R)) {
            if (right == nullptr) right = new node({0, inf});
            ans = min(ans, right->query_segment(x, m + 1, r, L, R));
        }
        return ans;
    }
};

```

```

struct LiChaoTree {
    int L, R;
    node* root;
    LiChaoTree() : L(numeric_limits<int>::min() / 2), R(
        numeric_limits<int>::max() / 2), root(nullptr) {}
    LiChaoTree(int L, int R) : L(L), R(R) { root = new node({0,
        inf}); }
    void add_line(Line line) { root->add_segment(line, L, R, L, R
        ); }
    void add_segment(Line line, int l, int r) { root->add_segment
        (line, L, R, l, r); }
    ll query(ll x) { return root->query_segment(x, L, R, L, R); }
};

```

```

    ll query_segment(ll x, int l, int r) { return root->
        query_segment(x, l, r, L, R); }
};

```

PersistentLiChaoTree.h

Description: Persistent/Dynamic Li Chao Tree for line minimum queries. Each update creates a new version (root).
Time: $\mathcal{O}(\log N)$ per update and query.

2d8279, 70 lines

```

struct Line {
    ll k, d;
    ll eval(ll x) { return k * x + d; }
};

struct LiChaoNode {
    Line line;
    int l, r;
    LiChaoNode() : line({0, (ll)2e18}), l(0), r(0) {}
    LiChaoNode(Line line) : line(line), l(0), r(0) {}
};

const int MAX_NODES = 5000005;
LiChaoNode t[MAX_NODES];
int T_cnt = 1;

int upd(int pre, Line nw, int l, int r) {
    int id = ++T_cnt;
    t[id] = t[pre];
    int m = l + (r - l) / 2;
    bool lef = nw.eval(l) < t[id].line.eval(l);
    bool mid = nw.eval(m) < t[id].line.eval(m);
    if (mid) swap(t[id].line, nw);
    if (l == r) return id;
    if (lef != mid) {
        if (!t[pre].l) {
            t[id].l = ++T_cnt;
            t[T_cnt] = LiChaoNode(nw);
        } else {
            t[id].l = upd(t[pre].l, nw, l, m);
        }
    } else {
        if (!t[pre].r) {
            t[id].r = ++T_cnt;
            t[T_cnt] = LiChaoNode(nw);
        } else {
            t[id].r = upd(t[pre].r, nw, m + 1, r);
        }
    }
    return id;
}

ll Query(int cur, int x, int l, int r) {
    if (!cur) return 2e18;
    ll val = t[cur].line.eval(x);
    if (l == r) return val;
    int m = l + (r - l) / 2;
    if (x <= m) val = min(val, Query(t[cur].l, x, l, m));
    else val = min(val, Query(t[cur].r, x, m + 1, r));
    return val;
}

struct PersistentLiChaoTree {
    int L, R;
    vector<int> roots;
    PersistentLiChaoTree(int L = -1e9, int R = 1e9) : L(L), R(R)
        {
            roots = {1};
        }
    void add_line(int version, Line line) {

```

```
        roots.push_back(upd(roots[version], line, L, R));
    }
    ll query(int version, int x) {
        return Query(roots[version], x, L, R);
    }
};

// PersistentLiChaoTree lt(-1e9, 1e9);
// lt.add_line({m1, c1}); // version 1
// lt.add_line({m2, c2}); // version 2
// ll res = lt.query(1, x); // Query version 1 at position x
```

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Time: $\mathcal{O}(\log N)$

1754b4, 53 lines

```
struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()) {}
    void recalc();
};

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }
```

template<class F> void each(Node* n, F f) {
 if (n) { each(n->l, f); f(n->val); each(n->r, f); }
}

```
pair<Node*, Node*> split(Node* n, int k) {
    if (!n) return {};
    if (cnt(n->l) >= k) { // "n->val" >= k" for lower_bound(k)
        auto [L,R] = split(n->l, k);
        n->l = R;
        n->recalc();
        return {L, n};
    } else {
        auto [L,R] = split(n->r, k - cnt(n->l) - 1); // and just "k"
        n->r = L;
        n->recalc();
        return {n, R};
    }
}

Node* merge(Node* l, Node* r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        return l->recalc(), l;
    } else {
        r->l = merge(l, r->l);
        return r->recalc(), r;
    }
}
```

```
Node* ins(Node* t, Node* n, int pos) {
    auto [l,r] = split(t, pos);
    return merge(merge(l, n), r);
}
```

// Example application: move the range [l, r] to index k

```
void move(Node*& t, int l, int r, int k) {
    Node *a, *b, *c;
    tie(a,b) = split(t, l); tie(b,c) = split(b, r - 1);
    if (k <= l) t = merge(ins(a, b, k), c);
    else t = merge(a, ins(c, b, k - r));
}
```

```
}

FenwickTree.h
Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and
updates single elements a[i], taking the difference between the old and new
value.
Time: Both operations are  $\mathcal{O}(\log N)$ .
```

e62fac, 22 lines

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >= 1) {
            if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
                pos += pw, sum -= s[pos-1];
        }
        return pos;
    }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h"157f07, 22 lines

```
struct FT2 {
    vector<vi> ys; vector<FT> ft;
    FT2(int limx) : ys(limx) {}
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
    }
    void init() {
        for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x-1].query(ind(x-1, y));
        return sum;
    }
};
```

RangeFenwickTree.h

Description: Fenwick Tree supporting Range Updates and Range Queries. update(l, r, v) adds v to all elements in [l,r]. query(l, r) returns the sum of elements in [l,r]. 0-indexed.

Time: $\mathcal{O}(\log N)$

8c056b, 22 lines

```
struct RangeFT {
    vector<ll> b1, b2;
    RangeFT(int n) : b1(n), b2(n) {}
}
```

```
void add(vector<ll>& b, int i, ll v) {
    for (; i < sz(b); i |= i + 1) b[i] += v;
}

ll sum(vector<ll>& b, int i) { // sum [0, i)
    ll res = 0;
    for (; i > 0; i &= i - 1) res += b[i-1];
    return res;
}

void update(int l, int r, ll v) { // add v to [l, r)
    add(b1, l, v); add(b1, r + 1, -v);
    add(b2, l, l * v); add(b2, r + 1, -(r + 1) * v);
}

ll query(int i) { // sum [0, i)
    return sum(b1, i) * i - sum(b2, i);
}

ll query(int l, int r) { // sum [l, r)
    return query(r + 1) - query(l);
}
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min($V[a], V[a + 1], \dots, V[b - 1]$) in constant time. Designed for half-open intervals $[a, b)$.

Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive); // [a, b)

Time: $\mathcal{O}(|V| \log |V| + Q)$

90b1a8, 21 lines

```
template<class T>
struct RMQ {
    vector<vector<T>>> jmp;
    vector<int> lg;
    RMQ() {}
    RMQ(const vector<T>& v) : jmp(1, v) {
        lg.assign(sz(v) + 1, 0);
        for(int i = 2; i<=sz(v); i++) lg[i] = lg[i / 2] + 1;
        for (int pw = 1, k = 1; pw * 2 <= sz(v); pw *= 2, ++k)
            jmp.emplace_back(sz(v) - pw * 2 + 1);
        rep(j, 0, sz(jmp[k]))
            jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
    T query(int a, int b) { // query on [a, b)
        if (a==b) return T();
        if (a>b) swap(a, b);
        int dep = lg[b-a];
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).

Time: $\mathcal{O}(N\sqrt{Q})$

a12ef4, 49 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer
```

```
vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
}
```

```
for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
}
return res;
}
```

```
vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) rep(end,0,2) {
        int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                    else { add(c, end); in[c] = 1; } a = c; }
        while (!(L[b] <= L[a] && R[a] <= R[b]))
            I[i++] = b, b = par[b];
        while (a != b) step(par[a]);
        while (i--) step(I[i]);
        if (end) res[qi] = calc();
    }
    return res;
}
```

FastMo.h
Description: Mo’s Algorithm using Hilbert Curve order. This improves cache locality and reduces the total pointer movement cost compared to standard block sorting. No block size constant is needed.
Time: $\mathcal{O}(N\sqrt{Q})$ for processing, sorting is $\mathcal{O}(Q \log Q \cdot \log N)$.2ff084, 29 lines

```
inline int64_t hilbert(int x, int y) {
    int64_t d = 0;
    for (int s = 1 << 20; s; s >= 1) { // 1<<20 covers N<= 10^6
        bool rx = x & s, ry = y & s;
        d += s * 1LL * s * ((3 * rx) ^ ry);
        if (ry) continue;
        if (rx) x = ~x, y = ~y;
        swap(x, y);
    }
    return d;
}

vi mo(vector<pii> Q) {
    int L = 0, R = 0;
    vi s(sz(Q)), res = s;
    iota(all(s), 0);
    sort(all(s), [&](int a, int b) {
        return hilbert(Q[a].first, Q[a].second) < hilbert(Q[b].first, Q[b].second);
    });
    for (int qi : s) {
        pii q = Q[qi];
```

```
        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }
    return res;
}
```

SqrtDecomp.h
Description: Generic Square Root Decomposition for point update and range query. Easily adjustable for min, max, gcd, etc. Modify the neutral element and the combine operation where indicated.
Time: $\mathcal{O}(\sqrt{N})$ 6e14e5, 38 lines

```
struct SqrtDecomp {
    static const int S = 450; // ~sqrt(N), adjust based on N
    // 450 is good for 2e5
    int n;
    vector<ll> a, b;

    SqrtDecomp(vector<ll> &z) : n(sz(z)), a(z), b(sz(z)/S+1) {
        for(int i=0;i<n;i++){
            b[i/S]+=a[i];
        }

        // Default example is Range Sum.
        void update(int p, ll val) {
            a[p] = val;
            int blk = p / S;
            b[blk] = 0; // 1. CHANGE HERE: Neutral element (e.g., INF for min, 0 for gcd)

            int start = blk * S;
            int end = min(start + S, n);
            rep(i, start, end) {
                b[blk] += a[i]; // 2. CHANGE HERE: Combine operation (e.g., min(b[blk], a[i]))
            }
        }

        ll query(int L, int R) {
            ll res = 0; // 3. CHANGE HERE: Neutral element
            int bL = L / S, bR = R / S;

            if (bL == bR) {
                rep(i, L, R + 1) res += a[i]; // 4. CHANGE HERE: Combine
            } else {
                rep(i, L, (bL + 1) * S) res += a[i]; // 4. CHANGE HERE: Combine
                rep(i, bL + 1, bR) res += b[i]; // 4. CHANGE HERE: Combine
                rep(i, bR * S, R + 1) res += a[i]; // 4. CHANGE HERE: Combine
            }
            return res;
        }
    };
};
```

Numerical (4)
4.1 Polynomials and recurrences
Polynomial.hc9b7b0, 17 lines

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h
Description: Finds the real roots to a polynomial.
Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$ b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin-1);
    dr.push_back(xmax+1);
    sort(all(dr));
    rep(i,0,sz(dr)-1) {
        double l = dr[i], h = dr[i+1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            rep(it,0,60) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
                else h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }
    return ret;
}
```

PolyInterpolate.h
Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k / (n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$ 08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
```

```
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
Time: $\mathcal{O}(N^2)$

"../number-theory/ModPow.h" 96548b, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i - j - 1]tr[j]$, given $S[0 \dots \geq n - 1]$ and $tr[0 \dots n - 1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is ϵ . Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.
Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);
Time: $\mathcal{O}(\log((b - a)/\epsilon))$

31d45b, 14 lines

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions.

8eecaf, 14 lines

```
typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
    pair<double, P> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

4756fc, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    rep(i,1,n*2)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage: double sphereVolume = quad(-1, 1, [](double x) { return quad(-1, 1, [&](double y) { return quad(-1, 1, [&](double z) { return x*x + y*y + z*z < 1; }}}}));

92dd79, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
```

```
d rec(F& f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}

template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b, x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

aa8530, 68 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
```

```
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
```

```
struct LPSolver {
    int m, n;
    vi N, B;
    vvd D;

    LPSolver(const vvd& A, const vd& b, const vd& c) :
        m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]; }
        rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m+1][n] = 1;
    }

    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
            T *b = D[i].data(), inv2 = b[s] * inv;
            rep(j,0,n+2) b[j] -= a[j] * inv2;
            b[s] = a[s] * inv2;
        }
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
}
```

```
bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
        int s = -1;
        rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
        if (D[x][s] >= -eps) return true;
        int r = -1;
        rep(i,0,m) {
            if (D[i][s] <= eps) continue;
            if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
```

```

        < MP[D[r][n+1] / D[r][s], B[r])) r = i;
    }
    if (r == -1) return false;
    pivot(r, s);
}

T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
        pivot(r, n);
        if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
        rep(i,0,m) if (B[i] == -1) {
            int s = 0;
            rep(j,1,n+1) ltj(D[i]);
            pivot(i, s);
        }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
}
};
```

4.3 Matrices

Gaussian.h
Description: Gaussian elimination with full pivoting. Reduces the first m columns of matrix a to Row Echelon Form (REF) or Reduced Row Echelon Form (RREF). Returns rank, determinant of the $m \times m$ square submatrix. p will store the column permutations if provided.
Time: $\mathcal{O}(n \cdot m \cdot \max(n,m))$

```

typedef vector<double> vd;
const double eps = 1e-12;
pair<int, double> gaussian(vector<vd>& a, int m, bool rref =
    true, vi* p = nullptr) {
    int n = sz(a), k = sz(a[0]), rank = 0;
    double det = 1;
    vi q(m); iota(all(q), 0);
    rep(i,0,m) {
        if (rank == n) break;
        int r = rank, c = i;
        rep(j,rank,n) rep(l,i,m)
            if (fabs(a[j][l]) > fabs(a[r][c])) r = j, c = l;
        if (fabs(a[r][c]) < eps) { det = 0; continue; }
        a[rank].swap(a[r]);
        rep(j,0,n) swap(a[j][i], a[j][c]);
        swap(q[i], q[c]);
        if (r != rank) det = -det;
        if (c != i) det = -det;
        det *= a[rank][i];
        double v = a[rank][i];
        rep(j,i,k) a[rank][j] /= v;
        rep(j,0,n) if (j != rank && (rref || j > rank)) {
            double f = a[j][i];
            rep(l,i,k) a[j][l] -= f * a[rank][l];
        }
        rank++;
    }
    if (p) *p = q;
    return {rank, det};
}
```

Determinant.h
Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

```
"Gaussian.h" bae227, 3 lines
```

```

double det(vector<vd>& a) {
    return gaussian(a, sz(a), false).second;
}

IntDeterminant.h
Description: Calculates determinant using modular arithmetics. Modulos
can also be removed to get a pure-integer version.
Time:  $\mathcal{O}(N^3)$ 
3313dc, 18 lines

const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h
Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

```
"Gaussian.h" 67d4bf, 10 lines
```

```

int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x);
    rep(i,0,n) A[i].push_back(b[i]);
    vi p;
    auto res = gaussian(A, m, true, &p);
    x.assign(m, 0);
    rep(i,res.first,n) if (fabs(A[i][m]) > eps) return -1;
    rep(i,0,res.first) x[p[i]] = A[i][m];
    return res.first;
}
```

SolveLinear2.h
Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```

"SolveLinear.h" 08e495, 7 lines

rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h
Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

```
fa2d7a, 34 lines
```

```

typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
```

```

        for (br=i; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if(b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
            A[j] ^= A[i];
        }
        rank++;
    }

x = bs();
for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
}
return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h
Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \bmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

```
"Gaussian.h" 856efa, 11 lines
```

```

int matInv(vector<vd>& A) {
    int n = sz(A);
    rep(i,0,n) rep(j,0,n) A[i].push_back(i == j);
    vi p;
    auto res = gaussian(A, n, true, &p);
    if (res.first < n) return res.first;
    vector<vd> resA(n, vd(n));
    rep(i,0,n) rep(j,0,n) resA[p[i]][j] = A[i][n + j];
    A = resA;
    return n;
}
```

MatrixFast.h
Description: Basic matrix multiplication and exponentiation modulo mod .
Time: $\mathcal{O}(N^3)$ for multiplication, $\mathcal{O}(N^3 \log b)$ for exponentiation.

```
7b7386, 20 lines
```

```

typedef vector<vector<ll>> Matrix;

Matrix mul(Matrix& a, Matrix& b, ll mod) {
    Matrix c(sz(a), vector<ll>(sz(b[0]), 0));
    rep(i,0,sz(a)) rep(j,0,sz(b[0])) rep(k,0,sz(b)) {
        c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % mod;
    }
    return c;
}
```

```

Matrix fastmat(Matrix a, ll b, ll mod) {
    Matrix res(sz(a), vector<ll>(sz(a[0]), 0));
    rep(i,0,sz(a)) res[i][i] = 1;
    while (b > 0) {
        if (b & 1) res = mul(res, a, mod);
        a = mul(a, a, mod);
        b >>= 1;
    }
```

```
    }
    return res;
}
```

Tridiagonal.h

Description: x = tridiagonal(d, p, q, b) solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, \, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\},$$

$$\{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: `fft(a)` computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , `reverse(start+1, end)`, FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
```

```
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
    }
    vd conv(const vd& a, const vd& b) {
        if (a.empty() || b.empty()) return {};
        vd res(sz(a) + sz(b) - 1);
        int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
        vector<C> in(n), out(n);
        copy(all(a), begin(in));
        rep(i,0,sz(b)) in[i].imag(b[i]);
        fft(in);
        for (C& x : in) x *= x;
        rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
        fft(out);
        rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
        return res;
    }
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

NumberTheoreticTransform.h

Description: `ntt(a)` computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(\text{mod}-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n , `reverse(start+1, end)`, NTT back. Inputs must be in $[0, \text{mod})$. the primitive root 62 can work with the following primes: 998244353, 167772161, 469762049, 1004535809, 1012924417.

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" de98fa, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s, mod)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
    }
    vl conv(const vl &a, const vl &b) {
        if (a.empty() || b.empty()) return {};
        int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
            n = 1 << B;
        int inv = modpow(n, mod - 2, mod);
        vl L(a), R(b), out(n);
        L.resize(n), R.resize(n);
        ntt(L), ntt(R);
        rep(i,0,n)
            out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
        ntt(out);
        return {out.begin(), out.begin() + s};
    }
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

d8c28b, 20 lines

```
#define int128 __int128_t
typedef vector<int128> vl;
typedef pair<int128, int128> pll;
void FST(vl& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
            int128 &u = a[j], &v = a[j + step]; tie(u, v) =
                // inv ? pll(v-u, u) : pll(v, u+v); // AND
                inv ? pll(v, u-v) : pll(u+v, u); // OR
            pll(u+v, u-v); XOR
        }
    }
    if (inv) for (int128& x : a) x /= sz(a); XOR only
}

vl conv(vl a, vl b) {
```

```
FST(a, 0); FST(b, 0);
rep(i,0,sz(a)) a[i] *= b[i];
FST(a, 1); return a;
}
```

SubsetConvolution.h

Description: Given f and g , computes $h[k] = \sum_{i \cup j = k, i \cap j = \emptyset} f[i]g[j]$.
Time: $\mathcal{O}(n^2 2^n)$ where n is the number of bits.

9642e3, 38 lines

const ll MOD = 998244353; // Or whatever the problem requires

```
void or_transform(vector<ll>& a, bool inv) {
    int n = sz(a);
    for (int len = 1; len < n; len <= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                if (!inv) {
                    a[i + len + j] += a[i + j];
                    if (a[i + len + j] >= MOD) a[i + len + j] -= MOD;
                } else {
                    a[i + len + j] -= a[i + j];
                    if (a[i + len + j] < 0) a[i + len + j] += MOD;
                }
            }
        }
    }
}
```

```
vector<ll> subset_conv(const vector<ll>& f, const vector<ll>& g, int n) {
    int N = 1 << n;
    vector<vector<ll>> fa(n + 1, vector<ll>(N, 0)), ga(n + 1, vector<ll>(N, 0));
    rep(i, 0, N) {
        fa[__builtin_popcount(i)][i] = f[i] % MOD;
        ga[__builtin_popcount(i)][i] = g[i] % MOD;
    }
    rep(i, 0, n + 1) {
        or_transform(fa[i], false);
        or_transform(ga[i], false);
    }
    vector<vector<ll>> ha(n + 1, vector<ll>(N, 0));
    rep(i, 0, n + 1) rep(j, 0, n + 1 - i) rep(mask, 0, N)
        ha[i + j][mask] = (ha[i + j][mask] + fa[i][mask] * ga[j][mask]) % MOD;
    rep(i, 0, n + 1) or_transform(ha[i], true);
    vector<ll> h(N);
    rep(i, 0, N) h[i] = ha[__builtin_popcount(i)][i];
    return h;
}
```

DirichletConv.h

Description: In-place Dirichlet Convolution (is_gcd=true for GCD, false for LCM). Computes $c[k] = \sum a[i] * b[j] \pmod{MOD}$. 1-based indexing assumed. Uses efficient addition/subtraction to avoid slow % operators.
Time: $\mathcal{O}(N \log N)$

2335a5, 26 lines

```
template<class T>
vector<T> dirichlet_conv(vector<T> a, vector<T> b, bool is_gcd, T mod) {
    int n = max(a.size(), b.size()) - 1;
    a.resize(n + 1); b.resize(n + 1);

    auto fwd = [&](vector<T>& v) {
        if (is_gcd)
            for (int i = 1; i <= n; ++i)
```

```
        for (int j = 2 * i; j <= n; j += i) { v[j] += v[j]; if (v[j] >= mod) v[j] -= mod; }
        else
            for (int i = n; i >= 1; --i)
                for (int j = 2 * i; j <= n; j += i) { v[j] += v[i]; if (v[j] >= mod) v[j] -= mod; }
    };

    fwd(a); fwd(b);
    for (int i = 1; i <= n; ++i) a[i] = 1LL * a[i] * b[i] % mod;

    if (is_gcd)
        for (int i = n; i >= 1; --i)
            for (int j = 2 * i; j <= n; j += i) { a[i] -= a[j]; if (a[i] < 0) a[i] += mod; }
        else
            for (int i = 1; i <= n; ++i)
                for (int j = 2 * i; j <= n; j += i) { a[j] -= a[i]; if (a[j] < 0) a[j] += mod; }

    return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

```
"Euclid.h"
35bfea, 18 lines

const ll mod = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll xx) : x(xx) {}
    Mod operator+(Mod b) { return Mod((x + b.x) % mod); }
    Mod operator-(Mod b) { return Mod((x - b.x + mod) % mod); }
    Mod operator*(Mod b) { return Mod((x * b.x) % mod); }
    Mod operator/(Mod b) { return *this * invert(b); }
    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, mod, x, y);
        assert(g == 1); return Mod((x + mod) % mod);
    }
    Mod operator^(ll e) {
        if (!e) return Mod(1);
        Mod r = *this ^ (e / 2); r = r * r;
        return e&1 ? *this * r : r;
    }
};
```

ModInverseEuclid.h

Description: Computes Modular Inverse using Extended Euclid.

"Euclid.h"1434ea, 5 lines

```
pair<bool, ll> modinv(ll a, ll m){
    ll x, y, g = euclid(a, m, x, y);
    if (g != 1) return {false, 0};
    return {true, (x % m + m) % m};
}
```

ModInverseSieve.h

Description: Pre-computation of modular inverses. Assumes $LIM \leq mod$ and that mod is a prime.

f0a7a3, 6 lines

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1;
void calcModInv(){
```

```
    inv[1] = 1;
    rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
}
```

ModPow.h

```
"ModMulLL.h"
7ad6ad, 6 lines

ll modpow(ll b, ll e, const ll mod) {
    ll ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
 $modsum(to, c, k, m) = \sum_{i=0}^{to-1} (k \cdot i + c) \% m$. $divsum$ is similar but for floored division. It is equivalent to counting lattice points (points with integer coordinates) under the line $y = \frac{k \cdot x + c}{m}$ in the first quadrant. $divsum(to, c, k, m) = \sum_{i=0}^{to-1} \left\lfloor \frac{k \cdot i + m}{c} \right\rfloor$
Time: $\log(m)$, with a large constant.

5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

CntLattice.h

Description: Counts non-negative integer solutions (x, y) to $ax + by \leq c$. Equivalent to counting lattice points in the triangle bounded by axes and the line. Requires 'divsum'.
Time: $\mathcal{O}(\log(\min(a, b)))$

"ModSum.h"1ce3a4, 6 lines

```
ll lattice_cnt(ll a, ll b, ll c) {
    if (c < 0) return 0;
    if (a == 0 || b == 0) return -1;
    ll n = c / a;
    return divsum(n + 1, c % a, a, b) + n + 1;
}
```

ModMulLL.h

Description: Calculate $a \cdot b \pmod c$ (or $a^b \pmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

a9c350, 9 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
```

```
// ull modmul(ull a, ull b, ull M) {
//     return (ull)((-int128)a * b % M);
// }
```

ModLog.h
Description: Returns the smallest $x > 0$ s.t. $a^x \equiv b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .
Time: $\mathcal{O}(\sqrt{m})$

```
11 modLog(11 a, 11 b, 11 m) {
    11 n = (11) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<11, 11> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSqrt.h
Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 \equiv a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
"ModPow.h"
19a793, 24 lines

11 sqrt(11 a, 11 p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    11 s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    11 x = modpow(a, (s + 1) / 2, p);
    11 b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        11 t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return x;
        11 gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

PrimitiveRoot.h
Description: Finds a primitive root modulo p .
Time: $\mathcal{O}(\text{ans} \cdot \log \phi(p) \cdot \log p)$

```
"ModPow.h"
63ac76, 10 lines

11 primitiveRoot(11 p) {
    if (p == 2) return 1;
    vector<11> factors = factor(p - 1); // Get prime factors of p - 1
    for (11 g = 2; ; ++g) {
        bool ok = 1;
        for (auto f : factors)
            if (modpow(g, (p - 1) / f.first, p) == 1) { ok = 0; break; }
        if (ok) return g;
    }
}
```

DiscreteRoot.h
Description: Solves $x^k \equiv a \pmod p$ for prime p . Returns one solution, or -1 if none exists. Requires 'modLog', 'primitiveRoot', 'linearCongruence'.

```
"PrimitiveRoot.h", "LinearCong.h", "modLog.h"
0691c2, 12 lines

11 discreteRoot(11 k, 11 a, 11 p) {
    if (a == 0) return 0;
    11 g = primitiveRoot(p);
    // 1. Solve g^y = a (mod p) for y (Discrete Log)
    11 y = modLog(g, a, p);
    if (y == -1) return -1;
    // 2. Solve k * x = y (mod p-1) for x (Linear Congruence)
    auto res = linearCong(k, y, p - 1); // Returns {x0, m}
    if (res.first == -1) return -1;
    // 3. Result is g^x (mod p)
    return modpow(g, res.first, p);
}
```

5.2 Primality

FastEratosthenes.h
Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i,0,min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

Eratosthenes.h
Description: Prime sieve for generating all primes up to a certain limit.
Time: $\mathcal{O}(LIM)$.

```
// O(N)
const int LIM = 5'000'000;
vi spf, primes;
void eratosthenes() {
    spf.resize(LIM);
    for (int i = 2; i < LIM; i++) {
        if (spf[i] == 0) {
            spf[i] = i;
            primes.push_back(i);
        }
        for (int j = 0; i * primes[j] < LIM; j++) {
            spf[i * primes[j]] = primes[j];
            if (primes[j] == spf[i]) break;
        }
    }
}
```

SegmentedSieves.h
Description: Segmented Sieve for Primes, Phi, Mu, and Tau (Divisor Count). Computes values for $[L, R]$ using precomputed small primes. rem[i] > 1 after processing indicates a prime factor > sqrt(R).

Usage: 1. Linear sieve primes up to $\sqrt{R \cdot \{MAX\}}$.
2. Set LIM $\geq$ max range length $(R - L + 1)$.
3. segSieve (Primes), segPhi (Totient), segMu (Mobius), segTau (Divisors).
Time: $\mathcal{O}\left((R - L) \log \log R + \sqrt{R}\right)$

```
const int LIM = 1e6+5;
vector<11> primes; // populate first
11 rem[LIM];
vector<bool> segSieve(11 L, 11 R) {
    11 lim = sqrt(R);
    vector<bool> isP(R - L + 1, 1);
    if (L == 1) isP[0] = 0;
    for (11 p : primes) {
        if (p > lim) break;
        for (11 j = max(p * p, (L + p - 1) / p * p); j <= R; j += p)
            isP[j - L] = 0;
    }
    return isP;
}

11 phi[LIM];
void segPhi(11 L, 11 R) {
    11 n = R - L + 1;
    iota(phi, phi + n, L);
    iota(rem, rem + n, L);
    for (11 p : primes) {
        if (p * p > R) break;
        11 start = (L + p - 1) / p * p;
        for (11 i = max(start, p * p) - L; i < n; i += p) {
            phi[i] -= phi[i] / p;
            while (rem[i] % p == 0) rem[i] /= p;
        }
    }
    rep(i, 0, n)
        if (rem[i] > 1) phi[i] -= phi[i] / rem[i];
}

int mu[LIM];
void segMu(11 L, 11 R) {
    int n = R - L + 1;
    fill(mu, mu + n, 1);
    iota(rem, rem + n, L);
    for (11 p : primes) {
        if (p * p > R) break;
        // Start at first multiple of p in range [L, R]
        for (11 i = (L + p - 1) / p * p - L; i < n; i += p) {
            if (rem[i] % p == 0) { // Optimization: check
                before expensive division loop
                int e = 0;
                while (rem[i] % p == 0) rem[i] /= p, e++;
                if (e > 1) mu[i] = 0;
                else if (mu[i]) mu[i] = -mu[i];
            }
        }
    }
    rep(i, 0, n)
        if (rem[i] > 1 && mu[i]) mu[i] = -mu[i];
}

11 tau[LIM];
void segTau(11 L, 11 R) {
    int n = R - L + 1;
    fill(tau, tau + n, 1);
    iota(rem, rem + n, L);

    for (11 p : primes) {
        if (p * p > R) break;
```

```
for (ll i = (L + p - 1) / p * p - L; i < n; i += p) {
    int e = 0;
    while (rem[i] % p == 0) rem[i] /= p, e++;
    if (e) tau[i] *= (e + 1);
}
}
rep(i, 0, n)
    if (rem[i] > 1) tau[i] *= 2;
}
```

Eratosthenes1e9.h

Description: Extremely fast Wheel Sieve (mod 30) + Segmented Sieve. Generates all primes up to N.
Time: 0.5s for $N = 10^9$.

2a1adb, 56 lines

```
vi sieve(int N) {
    const int Q = 17, L = 1 << 15, M = (N + 29) / 30;
    static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
    struct P { int p, pos[8] };
    int v = sqrt(N), vv = sqrt(v);
    vector<bool> isp(v + 1, 1);
    for (int i = 2; i <= vv; ++i) if (isp[i])
        for (int j = i * i; j <= v; j += i) isp[j] = 0;

    vi primes = {2, 3, 5};
    primes.reserve(int(N / log(N) * 1.1) + 5);

    vector<P> sprimes; size_t pbeg = 0; int prod = 1;
    for (int p = 7; p <= v; ++p) {
        if (!isp[p]) continue;
        if (p <= Q) prod *= p, pbeg++, primes.push_back(p);
        P pp = {p, {}};
        for (int t = 0; t < 8; ++t) {
            int j = (p <= Q) ? p : p * p;
            while (j % 30 != rs[t]) j += p << 1;
            pp.pos[t] = j / 30;
        }
        sprimes.push_back(pp);
    }

    vector<unsigned char> pre(prod, 0xFF);
    for (size_t pi = 0; pi < pbeg; ++pi) {
        auto pp = sprimes[pi]; int p = pp.p;
        for (int t = 0; t < 8; ++t) {
            const unsigned char m = ~(1 << t);
            for (int i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
        }
    }

    const int B = (L + prod - 1) / prod * prod;
    vector<unsigned char> blk(B);
    for (int beg = 0; beg < M; beg += B) {
        int end = min(M, beg + B);
        for (int i = beg; i < end; i += prod)
            copy(pre.begin(), pre.end(), blk.data() + (i - beg));
        if (beg == 0) blk[0] &= 0xFE;
        for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
            auto& pp = sprimes[pi]; p = pp.p;
            for (int t = 0; t < 8; ++t) {
                int i = pp.pos[t]; const unsigned char m = ~(1 << t);
                for (; i < end; i += p) blk[i - beg] &= m;
                pp.pos[t] = i;
            }
        }
        for (int i = beg; i < end; ++i)
            for (int m = blk[i - beg]; m > 0; m &= m - 1)
                primes.push_back(i * 30 + rs[__builtin_ctz(m)]);
    }
    while (primes.size() && primes.back() > N) primes.pop_back();
}
```

```
return primes;
}
```

ff802e, 7 lines

PrimesToFactors.h

```
Description: Generate all divisors from a list of (prime, exponent).
vector<ll> getDivisors(const vector<pll>& fact) {
    vector<ll> d = {1};
    for (auto [p, e] : fact)
        for (int i = 0, n = d.size(); i < n; ++i)
            for (ll v = d[i], j = 0; j < e; ++j) d.push_back(v * p);
    return d;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

60dcd1, 12 lines

```
"ModMulLL.h", "ModPow.h"
bool isPrime ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
            s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

PollardRho.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

d8d98d, 18 lines

```
"ModMulLL.h", "MillerRabin.h"
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

33ba8f, 5 lines

5.3 Divisibility

Euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
```

```
return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

04d93a, 7 lines

```
"Euclid.h"
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

LinearCong.h

Description: Solves $ax \equiv b \pmod n$. Returns $\{x_0, m\}$ such that the general solution is $x \equiv x_0 \pmod m$. To get all solutions in $[0, n)$: $x_k = x_0 + k \cdot m$ for $k \in [0, \gcd(a, n))$. Requires 'euclid' which returns $\{g, x, y\}$.
Time: $\mathcal{O}(\log n)$

a533c6, 7 lines

```
"Euclid.h"
pair<ll, ll> linearCong(ll a, ll b, ll n) {
    ll x, y, g = euclid(a, n, x, y);
    ll m = n / g;
    if (b % g) return {-1, -1};
    x = (x % m + m) % m; // Normalize x
    return {(x * (b / g)) % m, m};
}
```

LinearDiophantine.h

Description: Solves the Diophantine equation $ax + by = c$. Stores the solution in x and y such that $ax + by = c$. Returns false if no solution exists. To get all solcdns: $x = x_0 + k \cdot \frac{b}{g}, \quad y = y_0 - k \cdot \frac{a}{g}$. Requires 'euclid' (extended GCD).
Time: $\mathcal{O}(\log(\min(a, b)))$

92db6e, 11 lines

```
"Euclid.h"
bool linearDiophantine(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    if (!a && !b) return c == 0;
    ll x0, y0, gc = euclid(abs(a), abs(b), x0, y0); // |a|x0 +
        |b|y0 = g
    if (c % gc) return false;
    __int128 r = c / gc;
    x = (ll)(r * x0 % (b / gc)); // Normalize x for small
        magnitude
    if (a < 0) x = -x;
    y = (c - a * x) / b;
    g=gc;
    return true;
}
```

5.3.1 Number of solutions $ax+by=c$

General solution form is:

$$x = x_0 + k \cdot \frac{b}{g}, \quad y = y_0 - k \cdot \frac{a}{g}$$

We need $x > 0$ and $y > 0$.

That gives constraints on k :

$$k > \frac{-x_0 g}{b}, \quad k < \frac{y_0 g}{a}$$

5.3.2 Bézout’s identity

For $a \neq 0, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{gcd(a,b)}, y - \frac{ka}{gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler’s ϕ* function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n$ coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$.
If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$.
 $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Generalized Euler’s thm: $a^b \equiv a^{b \pmod{\phi(n)} + \phi(n)} \pmod n$, works for all a and n , provided $b \geq \phi(n)$.
Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.
Time: $\mathcal{O}(N \log \log N)$

```
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for (;) {
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1/(y - (d)a)) > 3*N) {
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `fracBS({}(Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`
Time: $\mathcal{O}(\log(N))$

```
27ab3e, 25 lines
```

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
            hi.p += lo.p * adv;
            hi.q += lo.q * adv;
            dir = !dir;
            swap(lo, hi);
            A = B; B = !adv;
        }
        return dir ? hi : lo;
    }
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$ (coprime), and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

Divisor Bound: The number of divisors $d(n)$ grows slowly.

$$\sum_{d|n} d = O(n \log \log n)$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 1344 for $n < 1e9$, 2000 for $n < 1e10$, 103680 for $n < 1e18$, 200 000 for $n < 1e19$. Approx bound:
 $d(n) \approx n^{1.066 / \ln \ln n}$.

5.8 Mobius Function

Mobius.h

Description: denoted as $\mu(n)$, is a fundamental number theory function that maps positive integers to $\{-1, 0, 1\}$ based on their prime factorization.
Time: $\mathcal{O}(N \log \log N)$

```
19a9c0, 12 lines
```

```
const int LIM = 1'000'000;
vi mu(LIM);
bitset<LIM> vis;
void mobius(){
    rep(i, 1, LIM) mu[i]=1;
    rep(i, 2, LIM) if(!vis[i]) // prime encountered
        for(int j=i; j<LIM; j+=i){
            vis[j]=1;
            if(j%(ll) i*i==0) mu[j]=0;
            else mu[j] *= -1;
        }
}
```

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1]$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f\left(\left\lfloor \frac{n}{m} \right\rfloor\right) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g\left(\left\lfloor \frac{n}{m} \right\rfloor\right)$$

5.9 Mega Sieve

MegaSieve.h

Description: Linear Sieve. Computes multiplicative functions for all numbers up to N . μ (Mobius), ϕ (Euler totient), num (number of divisors), and sig (sum of divisors). $cnt[i]$ stores the exponent of the smallest prime factor of i , and $sp[i]$ stores the sum of the geometric series of that prime factor.
Time: $\mathcal{O}(N)$

```
c5440d, 23 lines
```

```
const int N = 1e7+5;
int primes[N], k, mu[N], phi[N], num[N], sig[N], cnt[N], sp[N];
bitset<N> vis;

void megasieve() {
    mu[1] = phi[1] = num[1] = sig[1] = 1;
    for (int i = 2; i < N; ++i) {
        if (!vis[i]) primes[k++] = i, mu[i] = -1, phi[i] = i-1, num[i] = 2, sig[i] = sp[i] = i+1, cnt[i] = 1;
        for (int j = 0, P, x; j < k && (x = i * (P=primes[j])) < N; ++j) {
            vis[x] = 1;
            if (i % P) {
                mu[x] = -mu[i], phi[x] = phi[i] * (P - 1);
            }
        }
    }
}
```

```
cnt[x] = 1, num[x] = num[i] * 2;
sp[x] = P + 1, sig[x] = sig[i] * sp[x];
} else {
mu[x] = 0, phi[x] = phi[i] * P;
cnt[x] = cnt[i] + 1, num[x] = num[i] / (cnt[i] + 1) * (
    cnt[x] + 1);
sp[x] = sp[i] * P + 1, sig[x] = sig[i] / sp[i] * sp[x];
break;
}
}
```

5.10 Theorems

If $\gcd(A, B) = 1$, then $\gcd(A + B, A \cdot B) = 1$

5.10.1 Modular & Factorial

Legendre.h
Description: Legendre's Formula. Finds the largest x such that $k^x \mid n!$ where k is prime. For composite $k = k_1^{p_1} \dots k_m^{p_m}$, let a_i be the count of k_i in $n!$. The answer for composite k is $\min_i \lfloor a_i / p_i \rfloor$.
Time: $\mathcal{O}(\log_k n)$

```
ll fact_pow(ll n, ll k) {
    ll res = 0;
    while (n) {
        n /= k;
        res += n;
    }
    return res;
}
```

Wilson's Theorem:
 $(p - 1)! \equiv -1 \pmod p$ iff p is prime. Any integer $n > 1$ is prime iff $(n - 1)! + 1$ is divisible by n .

Legendre's Formula:
The exponent of prime p in $n!$ is $\sum_{k=1}^\infty \lfloor \frac{n}{p^k} \rfloor = \frac{n - S_p(n)}{p - 1}$, where $S_p(n)$ is the sum of digits of n in base p .

Lifting The Exponent (LTE):
If $p \mid x - y, p \nmid n, x, y$, then $v_p(x^n - y^n) = v_p(x - y) + v_p(n)$. If $p = 2, v_2(x^n - y^n) = v_2(x - y) + v_2(n) + v_2(x + y) - 1$ for even n .

5.10.2 Sums of Squares

Fermat (2 Squares):
 $n = a^2 + b^2$ iff $v_p(n)$ is even for every prime $p \equiv 3 \pmod 4$.

Legendre (3 Squares):
 $n = a^2 + b^2 + c^2$ iff $n \not\equiv 4^k(8m + 7)$.

Lagrange (4 Squares):
Every $n \in \mathbb{N}$ is the sum of 4 integer squares.

5.10.3 Diophantine & Sequences

Chicken McNugget:
For coprime a, b , the largest integer that cannot be written as $ax + by$ ($x, y \geq 0$) is $ab - a - b$. There are exactly $\frac{(a-1)(b-1)}{2}$ such integers.
Zeckendorf:

Every positive integer can be uniquely represented as a sum of non-consecutive Fibonacci numbers $(F_1 = 1, F_2 = 2, \dots)$.

Bertrand's Postulate:
For $n > 1$, there is always a prime p such that $n < p < 2n$.
Dirichlet:
If $\gcd(a, b) = 1$, the sequence $an + b$ contains infinitely many primes.

5.10.4 Conjectures

Goldbach (Strong):
Every even integer $n > 2$ is the sum of two primes.
Goldbach (Weak):
Every odd integer $n > 5$ is the sum of three primes.

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

$$\binom{n}{r} = \frac{n \times (n - 1) \times \dots \times (n - r + 1)}{r!}$$

$$P(n, r) = {}_nP_r = \frac{n!}{(n - r)!}$$

$$C(n, r) = {}_nC_r = \frac{n!}{r!(n - r)!}$$

$$C(n, r) = C(n, n - r)$$

Useful Formulas

$$\sum_{r=0}^n \binom{n}{r} = 2^n, \quad \sum_{r=0}^n (-1)^r \binom{n}{r} = 0$$

$$\binom{n}{0} + \binom{n}{2} + \binom{n}{4} + \dots = 2^{n-1}$$

$$\sum_{r=0}^n \binom{n}{r}^2 = \binom{2n}{n}, \quad \sum_{r=0}^n r \binom{n}{r} = n \cdot 2^{n-1}$$

$$\sum_{i=0}^n (i + 1) \binom{n}{i} = (n + 2)2^{n-1}$$

$$\sum_{k=0}^r \binom{m}{k} \binom{n}{r - k} = \binom{m + n}{r}$$

$$\sum_{i=k}^n \binom{i}{k} = \binom{n + 1}{k + 1}$$
$$\binom{n}{k} = \binom{n - 1}{k - 1} + \binom{n - 1}{k}$$

```
IntPerm.h
Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time:  $\mathcal{O}(n)$ 
044568, 6 lines
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n - 1)(D(n - 1) + D(n - 2)) = nD(n - 1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside's lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Stars and Bars

Splitting n items into k groups of unknown size can be represented as splitting n with k bars.

$$\binom{n + k - 1}{n} = \binom{n + k - 1}{k - 1}$$

6.2.2 Repeated Prefix Sums

In an infinite grid where the first row (row 0) is all ones ($A_{0,d} = 1$) and every subsequent row is the prefix sum of the row directly above it ($A_{m,d} = \sum_{c=0}^d A_{m-1,c}$), the cell at row $m - 1$ and column d is:

$$A_{m-1,d} = \binom{d+m-1}{d}$$

Combinatorially, this counts the number of valid grid paths from the origin (0,0) to $(m - 1, d)$ moving only right or down. By linearity, applying the prefix sum operation m times to an array propagates a value v at index i to index $j \geq i$ as:

$$v \cdot \binom{(j-i)+m-1}{j-i}$$

6.2.3 Tiling and Non-adjacent Selection

Number of ways to choose k non-adjacent items from n is $\binom{n-k+1}{k}$. Number of ways to tile a $1 \times n$ board with m dominoes (1×2) and $n - 2m$ squares (1×1) is $\binom{n-m}{m}$. For a circle of n items, choosing k non-adjacent items: $\frac{n}{n-k} \binom{n-k}{k}$. The sum $\sum_{m=0}^{\lfloor n/2 \rfloor} \binom{n-m}{m} = F_{n+1}$ (Fibonacci).

6.2.4 Pairing

Number of ways to group k (even) items into pairs:

$$(k-1)!! = 1 \cdot 3 \cdot 5 \dots (k-1) = \frac{k!}{(k/2)!2^{k/2}}$$

6.2.5 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

$\frac{n}{p(n)}$	0	1	2	3	4	5	6	7	8	9	20	50	100
	1	1	2	3	5	7	11	15	22	30	627	$\sim 2\text{e}5$	$\sim 2\text{e}8$

6.2.6 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

lucas.h
Description: Computes $\binom{n}{r} \pmod{p}$ for large n, r and small prime p . The formula is $\binom{n}{r} \equiv \prod \binom{n_i}{r_i} \pmod{p}$ where n_i, r_i are digits in base p . Useful when $n \geq p$. Requires a standard ‘nCr’ function for small inputs ($n < p$). p must be prime.
Time: $\mathcal{O}(\log_p n)$

f977be, 4 lines

```
11 lucas(11 n, 11 r, 11 p) {
    if (r == 0) return 1;
    return lucas(n / p, r / p, p) * nCr(n % p, r % p) % p;
}
```

6.2.7 Binomials

BinomialCoefficients.h
Description: Modular combinations and permutations. Precomputes factorials and inverses in linear time.
Time: $\mathcal{O}(N)$ init, $\mathcal{O}(1)$ query.

cd98e0, 14 lines

```
struct BinomialCoefficients {
    11 l, mod;
    vector<11> fact, inv, finv;
    BinomialCoefficients(11 n, 11 mod) : l(n), mod(mod), fact(n),
        inv(n), finv(n) {
        fact[0] = inv[1] = finv[0] = 1;
        for (int i = 1; i < n; i++) {
            fact[i] = fact[i - 1] * i % mod;
            if (i > 1) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
            finv[i] = finv[i - 1] * inv[i] % mod;
        }
    }
    11 nPr(11 n, 11 r){ return (n<r||r<0) ? 0:1LL * fact[n] *
        finv[n-r] % mod; }
    11 nCr(11 n, 11 r){ return 1LL * nPr(n, r) * finv[r] % mod;
    }
} bc(1e6+5, 1e9+7);
```

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1!k_2!\dots k_n!}$.

a0a312, 5 lines

```
11 multinomial(vi& v) {
    11 c = 1, m = v.empty() ? 1 : v[0];
    rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
    return c;
}
```

Binomial Theorem

The binomial theorem is a fundamental algebraic formula used to expand any binomial expression raised to a nonnegative integer power $(a + b)^n$.

$$(a + b)^n = \binom{n}{0} a^n b^0 + \binom{n}{1} a^{n-1} b^1 + \binom{n}{2} a^{n-2} b^2 + \dots + \binom{n}{n} a^0 b^n$$

$$(1 + x)^n = \binom{n}{0} + \binom{n}{1} x + \binom{n}{2} x^2 + \dots + \binom{n}{n} x^n$$

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
 $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n - 1, k - 1) + (n - 1)c(n - 1, k), \quad c(0, 0) = 1$$
$$\sum_{k=0}^n c(n, k) x^k = x(x + 1) \dots (x + n - 1)$$

$$c(8, k) = 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j + 1)$, $k + 1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n, k) = (n - k)E(n - 1, k - 1) + (k + 1)E(n - 1, k)$$

$$E(n, 0) = E(n, n - 1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n - 1, k - 1) + kS(n - 1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n + 1) \pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n - 2)! / ((d_1 - 1)! \dots (d_n - 1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \ C_{n+1} = \frac{2(2n+1)}{n+2}C_n, \ C_{n+1} = \sum C_iC_{n-i}$$
$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h
Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max |w_i| < \sim 2^{63}$.
Time: $\mathcal{O}(VE)$

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });

    int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
    rep(i,0,lim) for (Ed ed : eds) {
        Node cur = nodes[ed.a], &dest = nodes[ed.b];
        if (abs(cur.dist) == inf) continue;
        ll d = cur.dist + ed.w;
        if (d < dest.dist) {
            dest.prev = ed.a;
            dest.dist = (i < lim-1 ? d : -inf);
        }
    }
    rep(i,0,lim) for (Ed e : eds) {
        if (nodes[e.a].dist == -inf)
            nodes[e.b].dist = -inf;
    }
}
```

FloydWarshall.h
Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j]$ = inf if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or -inf if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
    int n = sz(m);
    rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
    rep(k,0,n) rep(i,0,n) rep(j,0,n)
```

```
    if (m[i][k] != inf && m[k][j] != inf) {
        auto newDist = max(m[i][k] + m[k][j], -inf);
        m[i][j] = min(m[i][j], newDist);
    }
    rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)
        if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
}
```

TopoSort.h
Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V| + |E|)$

```
vi topoSort(const vector<vi>& gr) {
    vi indeg(sz(gr)), q;
    for (auto& li : gr) for (int x : li) indeg[x]++;
    rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
    rep(j,0,sz(q)) for (int x : gr[q[j]])
        if (--indeg[x] == 0) q.push_back(x);
    return q;
}
```

7.2 Network flow

PushRelabel.h
Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(V^2\sqrt{E})$

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

    void addEdge(int s, int t, ll cap, ll rcap=0) {
        if (s == t) return;
        g[s].push_back({t, sz(g[t]), 0, cap});
        g[t].push_back({s, sz(g[s])-1, 0, rcap});
    }

    void addFlow(Edge& e, ll f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f; e.c -= f; ec[e.dest] += f;
        back.f -= f; back.c += f; ec[back.dest] -= f;
    }

    ll calc(int s, int t) {
        int v = sz(g); H[s] = v; ec[t] = 1;
        vi co(2*v); co[0] = v-1;
        rep(i,0,v) cur[i] = g[i].data();
        for (Edge& e : g[s]) addFlow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty()) if (!hi--) return -ec[s];
            int u = hs[hi].back(); hs[hi].pop_back();
            while (ec[u] > 0) // discharge u
                if (cur[u] == g[u].data() + sz(g[u])) {
                    H[u] = le9;
                    for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
                        H[u] = H[e.dest]+1, cur[u] = &e;
                }
        }
```

```
        if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
                --co[H[i]], H[i] = v + 1;
        hi = H[u];
    } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
        addFlow(*cur[u], min(ec[u], cur[u]->c));
    else ++cur[u];
}
}
bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

MinCostMaxFlow.h
Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.to });
                }
            }
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
```

```
ll totflow = 0, totcost = 0;
while (path(s), seen[t]) {
    ll fl = INF;
    for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

    totflow += fl;
    for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
    }
}
rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
return {totflow, totcost/2};
}
```

```
// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $O(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

482fe0, 36 lines

```
template<class T> T edmondsKarp(vector<unordered_map<int, T>>&
    graph, int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vi par(sz(graph)), q = par;

    for (;;) {
        fill(all(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        rep(i,0,ptr) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
    }
out:
    T inc = numeric_limits<T>::max();
    for (int y = sink; y != source; y = par[y])
        inc = min(inc, graph[par[y]][y]);

    flow += inc;
    for (int y = sink; y != source; y = par[y]) {
        int p = par[y];
        if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
        graph[y][p] += inc;
    }
}
```

```
}

MinCut.h
Description: After running max-flow, the left side of a min-cut from  $s$  to  $t$  is given by all vertices reachable from  $s$ , only traversing edges with positive residual capacity.
```

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}(V^3)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i,0,n) co[i] = {i};
    rep(ph,1,n) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it,0,n-ph) { //  $O(V^2) \rightarrow O(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i,0,n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i,0,n) mat[s][i] += mat[t][i];
        rep(i,0,n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    rep(i,1,N) {
        PushRelabel D(N); // Dinic also works
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j,i+1,N)
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}
```

7.3 Matching

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); hopcroftKarp(g, btoa);

Time: $\mathcal{O}(\sqrt{VE})$

f612e4, 42 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
    if (A[a] != L) return 0;
    A[a] = -1;
    for (int b : g[a]) if (B[b] == L + 1) {
```

```
        B[b] = 0;
        if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
            return btoa[b] = a, 1;
    }
    return 0;
}
```

```
int hopcroftKarp(vector<vi>& g, vi& btoa) {
    int res = 0;
    vi A(g.size()), B(btoa.size()), cur, next;
    for (;;) {
        fill(all(A), 0);
        fill(all(B), 0);
        cur.clear();
        for (int a : btoa) if (a != -1) A[a] = -1;
        rep(a,0,sz(g)) if (A[a] == 0) cur.push_back(a);
        for (int lay = 1;; lay++) {
            bool islast = 0;
            next.clear();
            for (int a : cur) for (int b : g[a]) {
                if (btoa[b] == -1) {
                    B[b] = lay;
                    islast = 1;
                }
                else if (btoa[b] != a && !B[b]) {
                    B[b] = lay;
                    next.push_back(btoa[b]);
                }
            }
            if (islast) break;
            if (next.empty()) return res;
            for (int a : next) A[a] = lay;
            cur.swap(next);
        }
        rep(a,0,sz(g))
            res += dfs(a, 0, g, btoa, A, B);
    }
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); dfsMatching(g, btoa);
Time: $\mathcal{O}(VE)$

522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}

int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i,0,sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h"	da4196, 20 lines
<pre>vi cover(vector<vi>& g, int n, int m) { vi match(m, -1); int res = dfsMatching(g, match); vector<bool> lfound(n, true), seen(m); for (int it : match) if (it != -1) lfound[it] = false; vi g, cover; rep(i,0,n) if (lfound[i]) q.push_back(i); while (!q.empty()) { int i = q.back(); q.pop_back(); lfound[i] = 1; for (int e : g[i]) if (!seen[e] && match[e] != -1) { seen[e] = true; q.push_back(match[e]); } } rep(i,0,n) if (!lfound[i]) cover.push_back(i); rep(i,0,m) if (seen[i]) cover.push_back(n+i); assert(sz(cover) == res); return cover; }</pre>	

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

	1e0fe9, 31 lines
<pre>pair<int, vi> hungarian(const vector<vi> &a) { if (a.empty()) return {0, {}}; int n = sz(a) + 1, m = sz(a[0]) + 1; vi u(n), v(m), p(m), ans(n - 1); rep(i,1,n) { p[0] = i; int j0 = 0; // add "dummy" worker 0 vi dist(m, INT_MAX), pre(m, -1); vector<bool> done(m + 1); do { // dijkstra done[j0] = true; int i0 = p[j0], j1, delta = INT_MAX; rep(j,1,m) if (!done[j]) { auto cur = a[i0 - 1][j - 1] - u[i0] - v[j]; if (cur < dist[j]) dist[j] = cur, pre[j] = j0; if (dist[j] < delta) delta = dist[j], j1 = j; } rep(j,0,m) { if (done[j]) u[p[j]] += delta, v[j] -= delta; else dist[j] -= delta; } j0 = j1; } while (p[j0]); while (j0) { // update alternating path int j1 = pre[j0]; p[j0] = p[j1], j0 = j1; } } rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1; return {-v[0], ans}; // min cost }</pre>	

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod .
Time: $\mathcal{O}(N^3)$

"./numerical/MatrixInverse-mod.h"	cb1912, 40 lines
<pre>vector<pii> generalMatching(int N, vector<pii>& ed) { vector<vector<ll>> mat(N, vector<ll>(N)), A; for (pii pa : ed) { int a = pa.first, b = pa.second, r = rand() % mod; mat[a][b] = r, mat[b][a] = (mod - r) % mod; } int r = matInv(A = mat), M = 2*N - r, fi, fj; assert(r % 2 == 0); if (M != N) do { mat.resize(M, vector<ll>(M)); rep(i,0,N) { mat[i].resize(M); rep(j,N,M) { int r = rand() % mod; mat[i][j] = r, mat[j][i] = (mod - r) % mod; } } } while (matInv(A = mat) != M); vi has(M, 1); vector<pii> ret; rep(it,0,M/2) { rep(i,0,M) if (has[i]) rep(j,i+1,M) if (A[i][j] && mat[i][j]) { fi = i; fj = j; goto done; } assert(0); done: if (fj < N) ret.emplace_back(fi, fj); has[fi] = has[fj] = 0; rep(sw,0,2) { ll a = modpow(A[fi][fj], mod-2); rep(i,0,M) if (has[i] && A[i][fj]) { ll b = A[i][fj] * a % mod; rep(j,0,M) A[i][j] = (A[i][j] - A[fj][j] * b) % mod; } swap(fi,fj); } } return ret; }</pre>	

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $\mathcal{O}(E + V)$

	fbef73, 26 lines
<pre>e vi val, comp, z, cont; int Time, ncomps; template<class G, class F> int dfs(int j, G& g, F& f) { int low = val[j] = ++Time, x; z.push_back(j); for (auto e : g[j]) if (comp[e] < 0) low = min(low, val[e] ?: dfs(e,g,f)); if (low == val[j]) { do { x = z.back(); z.pop_back(); comp[x] = ncomps; }</pre>	

<pre> cont.push_back(x); } while (x != j); f(cont); cont.clear(); ncomps++; } return val[j] = low; } template<class G, class F> void scc(G& g, F f) { int n = sz(g); val.assign(n, 0); comp.assign(n, -1); Time = ncomps = 0; rep(i,0,n) if (comp[i] < 0) dfs(i, g, f); }</pre>	
---	--

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: int eid = 0; ed.resize(N); for each edge (a,b) { ed[a].emplace_back(b, eid); ed[b].emplace_back(a, eid++); } bicomps([&](const vi& edgelist) {...});
Time: $\mathcal{O}(E + V)$

	c6b7e7, 32 lines
<pre>vi num, st; vector<vector<pii>> ed; int Time; template<class F> int dfs(int at, int par, F& f) { int me = num[at] = ++Time, top = me; for (auto [y, e] : ed[at]) if (e != par) { if (num[y]) { top = min(top, num[y]); if (num[y] < me) st.push_back(e); } else { int si = sz(st); int up = dfs(y, e, f); top = min(top, up); if (up == me) { st.push_back(e); f(vi(st.begin() + si, st.end())); st.resize(si); } else if (up < me) st.push_back(e); else { /* e is a bridge */ } } } return top; }</pre>	

<pre>template<class F> void bicomps(F f) { num.assign(sz(ed), 0); rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f); }</pre>	
---	--

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $a||b)&&(!a||c)&&(d||b)&&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

27439c, 62 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j) { // OR, at least one
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }

    void setValue(int x) { either(x, x); }
    void both(int f, int j) { setValue(f); setValue(j); } // AND
    void notBoth(int f, int j) { either(~f, ~j); } // NAND
    void neither(int f, int j) { setValue(~f); setValue(~j); } // NOR
    void setOnlyOne(int f, int j) { either(f, j); either(~f, ~j); } // XOR
    void bothAreSame(int f, int j) { either(~f, j); either(f, ~j); } // XNOR

    void atMostOne(const vi& li) { // NAND more than 2 variables (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        rep(i,2,sz(li)) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ? dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
```

```
        val.assign(2*N, 0); comp = val;
        rep(i,0,2*N) if (!comp[i]) dfs(i);
        rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end){ ret.push_back(x); s.pop_back(); continue; }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1; s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
    return {ret.rbegin(), ret.rend()};
}
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int& z = free[y] = 0; adj[y][z] != -1; z++);
    }
    rep(i,0,sz(eds))
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
    return ret;
}
```

```
}
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
    if (!P.any()) { if (!X.any()) f(R); return; }
    auto q = (P | X)._Find_first();
    auto cand = P & ~eds[q];
    rep(i,0,sz(eds)) if (cand[i]) {
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax)) return;
            q.push_back(R.back().i);
            vv T;
            for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk + 1].clear();
                    if (k < mnk) T[j++] .i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                rep(k,mnk,mxk + 1) for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
            }
        }
    }
}
```

```
        expand(T, lev + 1);
    } else if (sz(q) > sz(qmax)) qmax = q;
    q.pop_back(), R.pop_back();
}
}
vi maxClique() { init(V), expand(V); return qmax; }
Maxclique(vb connn) : e(connn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
}
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

BinaryLifting.h

Description: CALCULATE power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
    int on = 1, d = 1;
    while(on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    rep(i,1,d) rep(j,0,sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}
```

```
int jmp(vector<vi>& tbl, int nod, int steps){
    rep(i,0,sz(tbl))
        if(steps&(1<<i)) nod = tbl[i][nod];
    return nod;
}
```

```
int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 6c1825, 20 lines

```
struct LCA {
    int T = 0;
    vi time, path, ret, depth;
    RMQ<int> rmq;
    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)),
        depth(sz(C)) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            depth[y] = depth[v]+1;
            dfs(C, y, v);
        }
    }
};
```

```
    }
    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    int dist(int a, int b) {return depth[a] + depth[b] - 2*depth[
        lca(a,b)];}
};
```

LCAForest.h

Description: Data structure for computing lowest common ancestors in a forest in $\mathcal{O}(1)$.

Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 278946, 38 lines

```
struct LCAForest {
    vi euler, depth, first;
    RMQ<pii> rmq;

    LCAForest(int n, const vector<vi>& adj) : first(n, -1), depth
        (n) {
        for (int i = 0; i < n; i++) {
            if (first[i] == -1) dfs(adj, i, -1, 0);
        }
        vector<pii> ret(sz(euler));
        for (int i = 0; i < sz(euler); i++) {
            ret[i] = {depth[euler[i]], euler[i]};
        }
        rmq = RMQ<pii>(ret);
    }
    LCAForest(const vector<vi>& adj) : LCAForest(sz(adj), adj) {}

    void dfs(const vector<vi>& adj, int u, int p, int d) {
        first[u] = sz(euler);
        euler.push_back(u);
        depth[u] = d;
        for (int v : adj[u]) {
            if (v != p) {
                dfs(adj, v, u, d + 1);
                euler.push_back(u);
            }
        }
    }

    int lca(int a, int b) {
        int l = first[a], r = first[b];
        if (l > r) swap(l, r);
        return rmq.query(l, r + 1).second;
    }

    int dist(int a, int b) {
        return depth[a] + depth[b] - 2 * depth[lca(a, b)];
    }
};
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 9775a0, 21 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
```

```
    int m = sz(li)-1;
    rep(i,0,m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i,0,sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i,0,sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Time: $\mathcal{O}((\log N)^2)$

"../data-structures/DynamicSegmentTree.h" 9547af, 46 lines

```
template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    Node *tree;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N),pos(N),tree(new Node(0, N)){ dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (; v = par[rt[v]]) {
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree->add(l, r, val); });
    }
    int queryPath(int u, int v) { // Modify depending on problem
        int res = -1e9;
        process(u, v, [&](int l, int r) {
            res = max(res, tree->query(l, r));
        });
        return res;
    }
    int querySubtree(int v) { // modifySubtree is similar
```

```

    return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
}
};

```

HLD2.h

Description: Heavy-Light Decomposition. Supports path query decomposition.

Time: $\mathcal{O}(\log N)$ per path query

55c209, 53 lines

```

struct HLD {
    vi big, head, siz, parent, id, depth;
    int nxt, edge_based, root;
    HLD(int n, vector<vi> &adj, int r, int e=0)
        : big(n+1, -1), head(n+1), siz(n+1, 1), parent(n+1),
          id(n+1), depth(n+1), edge_based(e), root(r) {
        nxt = 0;
        head[root] = root;
        dfs(root, adj);
        flatten(root, adj);
    }
    void dfs(int u, vector<vi> &adj) {
        for(int v : adj[u]) {
            if(v == parent[u]) continue;
            parent[v] = u;
            depth[v] = depth[u] + 1;
            dfs(v, adj);
            siz[u] += siz[v];
            if(big[u] == -1 || siz[v] > siz[big[u]]) {
                big[u] = v;
            }
        }
    }
    void flatten(int u, vector<vi> &adj) {
        id[u] = nxt++;
        if(big[u] != -1) {
            head[big[u]] = head[u];
            flatten(big[u], adj);
        }
        for(int v : adj[u]) {
            if(v == parent[u] || v == big[u]) continue;
            head[v] = v;
            flatten(v, adj);
        }
    }
    vector<pii> path(int u, int v) {
        vector<pii> res;
        while(true) {
            if(head[u] == head[v]) {
                if(depth[v] > depth[u]) {
                    swap(u, v);
                }
                res.push_back({id[v] + edge_based, id[u]});
                return res;
            }
            if(depth[head[v]] > depth[head[u]]) {
                swap(u, v);
            }
            res.emplace_back(id[head[u]], id[u]);
            u = parent[head[u]];
        }
    }
};

```

CentroidDecomposition.h

Description: Centroid Decomposition generic template. Currently configured to count paths of length K . Run 'decomp()' once. For queries and updates, implement 'update()' and 'query()'. Can be modified to store $\log N$ ancestors/hubs per node.

Time: $\mathcal{O}(N \log N)$

4b690d, 53 lines

```

ll k, ans=0;
const int N = 2e5+5;
int sz[N], rem[N], cnt[N];
int cent_par[N]; // Stores the parent of each node in the
                  Centroid Tree
vector<int> adj[N];

int get_sz(int u, int p){ // get size of subtree of u
    sz[u]=1;
    for(int v:adj[u]){
        if (v != p && !rem[v]) sz[u] += get_sz(v, u);
    }
    return sz[u];
}
int get_centroid(int u, int p, int tot){ // get centroid of the
    subtree of u
    for(int v:adj[u]){
        if (v != p && !rem[v] && sz[v] > tot / 2) return
            get_centroid(v, u, tot);
    }
    return u;
}
void dfs_add(int u, int p, int dep, int delta){
    cnt[dep] += delta;
    for(int v:adj[u]){
        if(v==p || rem[v]) continue;
        dfs_add(v, u, dep+1, delta);
    }
}
void get_ans(int u, int p, int dep){
    if(dep<=k) ans += cnt[k-dep];
    for(int v:adj[u]){
        if(v==p || rem[v])continue;
        get_ans(v, u, dep+1);
    }
}
void decomp(int u, int p = -1){
    int comp_sz = get_sz(u, -1);
    int cent = get_centroid(u, -1, comp_sz);
    // get ans through current centroid
    cnt[0]=1;
    for(int v:adj[cent]){
        if(rem[v]) continue;
        get_ans(v, cent, 1);
        dfs_add(v, cent, 1, 1);
    }
    memset(cnt, 0, comp_sz*sizeof(int));
    // Save the centroid tree structure
    cent_par[cent] = p;
    // remove centroid and decompose
    rem[cent]=1;
    for(int v:adj[cent]){
        if(rem[v]) continue;
        decomp(v, cent);
    }
}

```

DominatorTree.h

Description: Computes the Dominator Tree for a directed graph (handles both general graphs and DAGs) from a given root.

Usage: DominatorTree DT(n, root, adj);

DT.idom[u] // Returns the immediate dominator of u (-1 if unreachable)

DT.tree[u] // Returns a vector of nodes immediately dominated by u

Time: $\mathcal{O}((V + E) \log V)$

1ce53e, 37 lines

```

struct DominatorTree {

```

```

    int timer = 0;
    vector<vi> tree, radj, bucket;
    vi idom, sdom, par, dsu, label, id, rev, dom;
    int find(int u, int x = 0) {
        if (u == dsu[u]) return x ? -1 : u;
        int v = find(dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[label[dsu[u]]] < sdom[label[u]]) label[u] =
            label[dsu[u]];
        return dsu[u] = v, x ? v : label[u];
    }
    void dfs(int u, const vector<vi>& g) {
        id[u] = ++timer;
        label[timer] = sdom[timer] = dsu[timer] = timer;
        rev[timer] = u;
        for (int v : g[u]) {
            if (!id[v]) dfs(v, g), par[id[v]] = id[u];
            radj[id[v]].push_back(id[u]);
        }
    }
    DominatorTree(int n, int root, const vector<vi>& g) :
        tree(n), radj(n+1), bucket(n+1), idom(n, -1), sdom(n+1),
        , par(n+1),
        dsu(n+1), label(n+1), id(n, 0), rev(n+1), dom(n+1) {
        dfs(root, g);
        for (int i = timer; i >= 1; i--) {
            for (int p : radj[i]) sdom[i] = min(sdom[i], sdom[
                find(p)]);
            if (i > 1) bucket[sdom[i]].push_back(i);
            for (int w : bucket[i]) dom[w] = (sdom[find(w)] ==
                sdom[w] ? sdom[w] : find(w));
            if (i > 1) dsu[i] = par[i];
        }
        for (int i = 2; i <= timer; i++) {
            if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
            tree[idom[rev[i]]] = rev[dom[i]].push_back(rev[i]);
        }
        if (timer > 0) idom[root] = root;
    }
};

```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

0fb462, 90 lines

```

struct Node { // Splay tree. Root's pp contains tree's parent.
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    Node() { c[0] = c[1] = 0; fix(); }
    void fix() {
        if (c[0]) c[0]->p = this;
        if (c[1]) c[1]->p = this;
        // (+ update sum of subtree elements etc. if wanted)
    }
    void pushFlip() {
        if (!flip) return;
        flip = 0; swap(c[0], c[1]);
        if (c[0]) c[0]->flip ^= 1;
        if (c[1]) c[1]->flip ^= 1;
    }
    int up() { return p ? p->c[1] == this : -1; }
    void rot(int i, int b) {
        int h = i ^ b;
        Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
        if ((y->p = p)) p->c[up()] = y;
        c[i] = z->c[i ^ 1];
        if (b < 2) {

```

```

    x->c[h] = y->c[h ^ 1];
    y->c[h ^ 1] = x;
}
z->c[i ^ 1] = this;
fix(); x->fix(); y->fix();
if (p) p->fix();
swap(pp, y->pp);
}
void splay() {
    for (pushFlip(); p; ) {
        if (p->p) p->p->pushFlip();
        p->pushFlip(); pushFlip();
        int c1 = up(), c2 = p->up();
        if (c2 == -1) p->rot(c1, 2);
        else p->p->rot(c2, c1 != c2);
    }
}
Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
}
};

struct LinkCut {
    vector<Node> node;
    LinkCut(int N) : node(N) {}

    void link(int u, int v) { // add an edge (u, v)
        assert(!connected(u, v));
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) { // remove an edge (u, v)
        Node *x = &node[u], *top = &node[v];
        makeRoot(top); x->splay();
        assert(top == (x->pp ? x->c[0]));
        if (x->pp) x->pp = 0;
        else {
            x->c[0] = top->p = 0;
            x->fix();
        }
    }
    bool connected(int u, int v) { // are u, v in the same tree?
        Node* nu = access(&node[u])->first();
        return nu == access(&node[v])->first();
    }
    void makeRoot(Node* u) {
        access(u);
        u->splay();
        if(u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0;
            u->fix();
        }
    }
}
Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
        pp->splay(); u->pp = 0;
        if (pp->c[1]) {
            pp->c[1]->p = 0; pp->c[1]->pp = pp; }
        pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
}
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

../../data-structures/UnionFindRollback.h
 39e620, 60 lines

```

struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node* &a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1,-1}), comp;
    deque<tuple<int, int, vector<Edge>>> cycs;
    rep(s,0,n) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1,{} };
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cycs.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
    }

    for (auto& [u,t,comp] : cycs) { // restore sol (optional)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto& e : comp) in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    rep(i,0,n) par[i] = in[i].a;
    return {res, par};
}

```

TreeIsomorphism.h

Description: Rooted unordered tree isomorphism using the Aho-Hopcroft-Ullman (AHU) algorithm. Computes a canonical integer ID for a specific rooted structure. The ID map is kept global for memoization across multiple test cases.

Time: $\mathcal{O}(N \log N)$ per test case.

150908, 26 lines

```

struct RootedTreeIso {
    map<vi, int> canonical_id;
    vi data;
    int id=0;

    int get_id(vi& children) {
        sort(children.begin(), children.end()); // remove
            sorting if ordered
        auto [it, inserted] = canonical_id.try_emplace(children
            , id);
        if (inserted) id++;
        return it->second;
    }

    int get_rooted_id(int u, int p, const vector<vi>& adj) {
        vi c;
        for (int v : adj[u]) {
            if (v != p) c.push_back(get_rooted_id(v, u, adj));
        }
        return data[u]=get_id(c);
    }

    vi compute(int root, const vector<vi>& adj) {
        data.assign(sz(adj), -1);
        get_rooted_id(root, -1, adj);
        return data;
    }
} tree_iso;

```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

7.8.3 Hammer-Simeone Theorem

A graph is **split** (vertices can be partitioned into a clique and an independent set) iff its degree sequence $d_1 \geq d_2 \geq \dots \geq d_n$ satisfies:

$$\sum_{i=1}^m d_i = m(m-1) + \sum_{i=m+1}^n d_i$$

where m is the largest i such that $d_i \geq i-1$.

- The first m vertices form the clique (in some realization).

- Complexity: $O(N \log N)$ to sort, $O(N)$ to check.
- Useful for problems involving chordal graphs (split $\mathbb{C}$ chordal).

7.8.4 Planar Graph Girth

Any simple planar graph with minimum degree ≥ 3 has girth $g \leq 5$. If it has no C_3 (triangles) or C_4 (4-cycles), the girth is exactly 5. By orienting undirected edges from lower degree to higher degree (breaking ties arbitrarily), the out-degree of any vertex in the directed graph is bounded by 5. C_3 and C_4 can then be verified in $O(V)$ time.

PlanarGirth.h

Description: Finds the girth of a planar graph with minimum degree ≥ 3 using Euler's limit. The girth of such a graph is at most 5. *dir_adj* must orient undirected edges from lower degree to higher degree (out-degree bounded by 5).

Time: $O(V)$

```
5ce106, 26 lines
int planarGirth(int n, const vector<vi>& adj, const vector<vi>&
    dir_adj) {
    int ans = 5;
    vi mark(n + 1), common(n + 1);

    // C3 Verification
    for(int u = 1; u <= n; u++) {
        for(int v : adj[u]) mark[v] = u;
        for(int v : dir_adj[u]) {
            for(int w : dir_adj[v]) {
                if(mark[w] == u) ans = min(ans, 3);
            }
        }
    }

    // C4 Verification
    for(int u = 1; u <= n; u++) {
        for(int v : adj[u]) {
            for(int w : dir_adj[v]) {
                if(w == u) continue;
                if(common[w] == u) ans = min(ans, 4);
                else common[w] = u;
            }
        }
    }
    return ans;
}
```

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

328417, 38 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
```

```
P operator*(T d) const { return P(x*d, y*d); }
P operator/(T d) const { return P(x/d, y/d); }
T dot(P p) const { return x*p.x + y*p.y; }
T cross(P p) const { return x*p.y - y*p.x; }
// positive -> b on the left of a, ccw from a to b
T cross(P a, P b) const { return (a-*this).cross(b-*this); }
T dist2() const { return x*x + y*y; }
double dist() const { return sqrt((double)dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
// calculates the smallest unsigned angle of ABC (*this is B)
// returns a value in the interval [0, pi]
double angle(P A, P C) const {
    P ba = A - *this;
    P bc = C - *this;
    return abs(atan2((double)ba.cross(bc), (double)ba.dot(bc)));
}
// from polar: P(r*cos(a), r*sin(a))
P unit() const { return *this/dist(); } // makes dist()==1
P perp() const { return P(-y, x); } // rotates +90 degrees ccw
P normal() const { return perp().unit(); }
T half() const { return y > 0 || (y == 0 && x > 0) ? 1 : 0; }
// returns point rotated 'a' radians ccw around the origin
P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << ", " << p.y << ")"; }
};
```

Line.h

Description: Line struct for geometry.

"Point.h" a82d4c, 19 lines

```
template<class T>
struct Line {
    typedef Point<T> P;
    P a, b;
    Line(P a, P b) : a(a), b(b) {}
    P v() const { return b - a; }
    // Signed distance from line ab to point p.
    // Positive on left side, negative on right.
    double dist(P p) const { return (double)v().cross(p - a) / v().dist(); }
    T dist2(P p) const { T c = v().cross(p - a); return c * c / v().dist2(); }
    int sideOf(P p) const { return sgn(v().cross(p - a)); }
    P proj(P p) const { return a + v() * v().dot(p - a) / v().dist2(); }
    P refl(P p) const { return proj(p) * 2 - p; }
    pair<int, P> inter(Line l) const {
        auto d = v().cross(l.v());
        if (d == 0) return {(a.cross(b, l.a) == 0), P(0, 0)};
        return {1, (a * l.a.cross(b, l.b) + b * l.a.cross(l.b, a)) / d};
    }
};
```

Segment.h

Description: Segment struct for geometry.

"Point.h", "OnSegment.h" 18eb91, 28 lines

```
template<class T>
struct Segment {
    typedef Point<T> P;
    P s, e;
    Segment(P s, P e) : s(s), e(e) {}
    double dist(P p) const {
        if (s == e) return (p - s).dist();
```

```
auto d = (e - s).dist2(), t = min(d, max{(T)0, (p - s).dot(e - s)}));
return ((p - s) * d - (e - s) * t).dist() / d;
}
bool on(P p) const { return onSegment(s, e, p); }
vector<P> inter(Segment o) const {
    auto oa = o.s.cross(o.e, s), ob = o.s.cross(o.e, e),
        oc = s.cross(e, o.s), od = s.cross(e, o.e);
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(s * ob - e * oa) / (ob - oa)};
    set<P> res;
    if (on(o.s)) res.insert(o.s);
    if (on(o.e)) res.insert(o.e);
    if (o.on(s)) res.insert(s);
    if (o.on(e)) res.insert(e);
    return {res.begin(), res.end()};
}
double dist(Segment o) const {
    if (!inter(o).empty()) return 0;
    return min({dist(o.s), dist(o.e), o.dist(s), o.dist(e)});
}
};
```

lineDistance.h

Description:

Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double)(b-a).cross(p-a) / (b-a).dist();
}
```

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);

bool onSegment = segDist(a,b,p) < 1e-10;

"Point.h" 5c88f4, 6 lines

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentToSegmentDistance.h

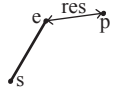
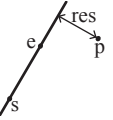
Description: Returns the shortest distance between two line segments.

Time: $O(1)$

"SegmentDistance.h", "SegmentIntersection.h" 5886f8, 5 lines

```
double segSegDist(P a, P b, P c, P d) {
    if (!segInter(a, b, c, d).empty()) return 0;
    return min({segDist(a, b, c), segDist(a, b, d),
        segDist(c, d, a), segDist(c, d, b)});
}
```

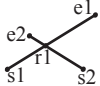
SegmentIntersection.h



Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
          oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

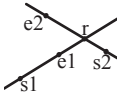


lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```



sideOf.h

Description: Returns where *p* is as seen from *s* towards *e*. 1/0/-1 ⇔ left/on line/right. If the optional argument *eps* is given 0 is returned if *p* is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;
"Point.h" 3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

"Point.h" c597e8, 3 lines

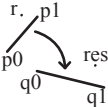
```
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

"Point.h" 03a306, 6 lines

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```



LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow. rayReflect calculates the outgoing direction vector of a velocity vector *v*_i*n* bouncing off line ab.

"Point.h" 58f3c6, 10 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
    P v = b - a;
    return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

```
template<class P>
P rayReflect(P a, P b, P v_in) {
    return lineProj(P(), b - a, v_in, true);
}
```

Angle.h
Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

0f0602, 35 lines

```
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};

bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
           make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
```

```
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ?
        make_pair(a, b) : make_pair(b, a.t360()));
}

Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleIntersectionArea.h

Description: Area of intersection of two circles.

"Point.h" 3b83e6, 13 lines

```
double circleIntersectionArea(Point<double> c1, double r1,
    Point<double> c2, double r2) {
    double d2 = (c1 - c2).dist2(), d = sqrt(d2);
    if (r1 + r2 <= d) return 0;
    if (d <= abs(r1 - r2)) {
        double r = min(r1, r2);
        return PI * r * r;
    }
    auto sectorArea = [](double r, double otherR, double d) {
        double phi = 2 * acos((r * r + d * d - otherR * otherR) /
            (2 * r * d));
        return 0.5 * r * r * (phi - sin(phi));
    };
    return sectorArea(r1, r2, d) + sectorArea(r2, r1, d);
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
```

```
vector<pair<P, P>> out;
for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
}
if (h2 == 0) out.pop_back();
return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

```
"Point.h" 20cfba, 9 lines
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
    P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
    double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    P h = ab.unit() * sqrt(h2);
    return {p - h, p + h};
}
```

CircleSegment.h

Description: Finds the intersection between a circle and a segment. Returns a vector of either 0, 1, or 2 intersection points.

```
"CircleLine.h" 6993be, 11 lines
template<class P>
vector<P> circleSegment(P c, double r, P a, P b) {
    vector<P> pts = circleLine(c, r, a, b);
    vector<P> res;
    for (P p : pts) {
        if ((a - p).dot(b - p) <= 1e-9) {
            res.push_back(p);
        }
    }
    return res;
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

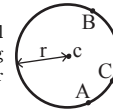
Time: $\mathcal{O}(n)$

```
"../../../../content/geometry/Point.h" 19add1, 19 lines
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = q + d * (t-1);
        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

circumcircle.h

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



```
"Point.h" 1caa3a, 9 lines
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()*(C-B).dist()*(A-C).dist() /
        abs((B-A).cross(C-A)) / 2;
}
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

Incenter.h

Description: Returns the incenter and inradius of a triangle.

```
"Point.h" 2dc0c5, 7 lines
template<class P>
pair<P, double> incenter(P a, P b, P c) {
    double la = (b - c).dist(), lb = (a - c).dist(), lc = (a - b).dist();
    P res = (a * la + b * lb + c * lc) / (la + lb + lc);
    double area = abs((b - a).cross(c - a)) / 2;
    return {res, 2 * area / (la + lb + lc)};
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

```
"circumcircle.h" 09dd0a, 17 lines
pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

8.3 Polygons

CheckCCW.h

Description: Returns true if the polygon vertices are in strictly counter-clockwise order using the Shoelace formula (signed area).

Time: $\mathcal{O}(N)$

```
"Point.h" 4282c7, 7 lines
template<class T>
bool isCounter(const vector<Point<T>>& p) {
    if (sz(p) < 3) return false;
    T a = p.back().cross(p[0]);
    rep(i, 0, sz(p) - 1) a += p[i].cross(p[i + 1]);
    return a > 0;
}
```

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

```
"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
"Point.h" f12300, 6 lines
template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PicksTheorem.h

Description: Pick's Theorem: $A = I + B/2 - 1$, where A is the area, I is the number of interior lattice points, and B is the number of boundary lattice points. For boundary points $B = \sum \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$.

```
"Point.h" 657d6c, 13 lines
ll countBoundaryPoints(const vector<Point<ll>>& v) {
    ll b = 0;
    rep(i,0,sz(v)) {
        Point<ll> d = v[i] - v[(i + 1) % sz(v)];
        b += gcd(abs(d.x), abs(d.y));
    }
    return b;
}

// I = (2A - B + 2) / 2
ll countInteriorPoints(ll area2, ll b) {
    return (abs(area2) - b + 2) / 2;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

```
"Point.h" 9706dc, 9 lines
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h

Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));

"Point.h"

d07181, 13 lines

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;
    rep(i,0,sz(poly)) {
        P cur = poly[i], prev = i ? poly[i-1] : poly.back();
        auto a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a < 0) != (b < 0))
            res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.push_back(cur);
    }
    return res;
}
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)

Time: $\mathcal{O}(N^2)$, where N is the total number of points

"Point.h", "sideOf.h"

3931c6, 33 lines

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
    double ret = 0;
    rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
        P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
        vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
        rep(j,0,sz(poly)) if (i != j) {
            rep(u,0,sz(poly[j])) {
                P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
                int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
                if (sc != sd) {
                    double sa = C.cross(D, A), sb = C.cross(D, B);
                    if (min(sc, sd) < 0)
                        segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
                } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0) {
                    segs.emplace_back(rat(C - A, B - A), 1);
                    segs.emplace_back(rat(D - A, B - A), -1);
                }
            }
        }
        sort(all(segs));
        for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
        double sum = 0;
        int cnt = segs[0].second;
        rep(j,1,sz(segs)) {
            if (!cnt) sum += segs[j].first - segs[j - 1].first;
            cnt += segs[j].second;
        }
        ret += A.cross(B) * sum;
    }
    return ret / 2;
}
```

ConvexHull.h

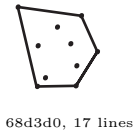
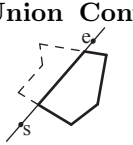
Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"

68d3d0, 17 lines



```
typedef Point<ll> P;
vector<P> convexHull(vector<P>& pts, bool include_collinear = false) {
    if (pts.size() <= 1) return pts;
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end(), pts.end()));
    vector<P> h(pts.size() * 2);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(pts.begin(), pts.end())) {
        for (auto& p : pts) {
            while (t >= s + 2 && (include_collinear ?
                h[t-2].cross(h[t-1], p) < 0 :
                h[t-2].cross(h[t-1], p) <= 0)) t--;
            h[t++] = p;
        }
    }
    return {h.begin(), h.begin() + t - (t > 1 && h[0] == h[t-1])};
}
```

"Point.h"

c571b8, 12 lines

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

"Point.h"

c571b8, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j)
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

"Point.h"

3b3ea8, 31 lines

AntipodalPairs.h

Description: Returns all unique antipodal pairs of a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n \log n)$ due to sorting, or $\mathcal{O}(n)$ if sorted output not needed.

"Point.h"

3b3ea8, 31 lines

```
typedef Point<ll> P;
vector<pair<int, int>> antipodalPairs(vector<P> &S) {
    int n = sz(S);
    if (n < 2) return {};
    if (n == 2) return {{0, 1}};
    vector<pair<int, int>> pairs;
    auto add = [&](int u, int v) {
        if (u > v) swap(u, v);
        pairs.push_back({u, v});
    };
    int j = 1;
    rep(i,0,n) {
        while (true) {
            ll diff = (S[(i + 1) % n] - S[i])
                .cross(S[(j + 1) % n] - S[j]);
            if (diff > 0) j = (j + 1) % n;
            else break;
        }
        add(i, j);
        add((i + 1) % n, j);
        ll diff = (S[(i + 1) % n] - S[i])
            .cross(S[(j + 1) % n] - S[j]);
        if (diff == 0) {
```

```
            add(i, (j + 1) % n);
            add((i + 1) % n, (j + 1) % n);
        }
    }
    sort(all(pairs));
    pairs.erase(unique(all(pairs)), pairs.end());
    return pairs;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"

71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

"Point.h"

7cf45b, 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}
```

```
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i,0,2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
```

```
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
}
if (res[0] == res[1]) return {res[0], -1};
if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
        case 0: return {res[0], res[0]};
        case 2: return {res[1], res[1]};
    }
return res;
}
```

MinkowskiSum.h
Description: Minkowski sum of two convex polygons. Both must be CCW. Returns a CCW polygon.
Time: $\mathcal{O}(n + m)$

"Point.h"	7a1afb, 19 lines
<pre>template<class P> vector<P> minkowskiSum(vector<P> a, vector<P> b) { auto reorder = [&](vector<P>& p) { int pos = 0; rep(i,1,sz(p)) if (p[i].y < p[pos].y (p[i].y == p[pos].y && p[i].x < p[pos].x)) pos = i; rotate(p.begin(), p.begin() + pos, p.end()); }; if (a.empty() b.empty()) return {}; reorder(a); reorder(b); int n = sz(a), m = sz(b), i = 0, j = 0; vector<P> res; while (i < n j < m) { res.push_back(a[i % n] + b[j % m]); auto c = (a[(i + 1) % n] - a[i % n]).cross(b[(j + 1) % m] - b[j % m]); if (c >= 0 && i < n) i++; if (c <= 0 && j < m) j++; } return res; }</pre>	

HalfPlaneIntersection.h
Description: Half-plane intersection. Each line (s, e) represents the half-plane to the left of s->e. Returns vertices of the intersection polygon in CCW order. The template assumes the region is perfectly closed, if not, you should artificially close the universe by pushing four massive half-planes representing a bounding box.
Time: $\mathcal{O}(N \log N)$

"Point.h", "lineIntersection.h"	36f09c, 33 lines
<pre>struct Halfplane { Point<double> s, e; double ang; Halfplane() {} Halfplane(Point<double> _s, Point<double> _e) : s(_s), e(_e) { ang = atan2((e - s).y, (e - s).x); } bool out(Point<double> p) { return (e - s).cross(p - s) < -1e-9; } bool operator<(const Halfplane& b) const { if (abs(ang - b.ang) > 1e-9) return ang < b.ang; return (e - s).cross(b.s - s) < -1e-9; } };</pre>	

```
vector<Point<double>> halfPlaneIntersection(vector<Halfplane> h
) {
    sort(all(h));
    deque<Halfplane> dq;
    rep(i,0,sz(h)) {
```

```
        if (i && abs(h[i].ang - h[i-1].ang) < 1e-9) continue;
        while (sz(dq) >= 2 && h[i].out(lineInter(dq[sz(dq)-2].s, dq
            [sz(dq)-2].e, dq.back().s, dq.back().e).second))
            dq.pop_back();
        while (sz(dq) >= 2 && h[i].out(lineInter(dq[0].s, dq[0].e,
            dq[1].s, dq[1].e).second))
            dq.pop_front();
        dq.push_back(h[i]);
    }
    while (sz(dq) >= 3 && dq[0].out(lineInter(dq[sz(dq)-2].s, dq[
        sz(dq)-2].e, dq.back().s, dq.back().e).second))
        dq.pop_back();
    while (sz(dq) >= 3 && dq.back().out(lineInter(dq[0].s, dq[0].
        e, dq[1].s, dq[1].e).second))
        dq.pop_front();
    if (sz(dq) < 3) return {};
    vector<Point<double>> res;
    rep(i,0,sz(dq)) res.push_back(lineInter(dq[i].s, dq[i].e, dq
        [(i+1)%sz(dq)].s, dq[(i+1)%sz(dq)].e).second);
    return res;
}
```

ConvexPolygonIntersection.h
Description: Returns the vertices of the intersection of two convex poly-gons. Both must be CCW.
Time: $\mathcal{O}(N \cdot M)$

"PolygonCut.h"	87a9d9, 8 lines
<pre>typedef Point<double> P; vector<P> convexPolygonIntersection(vector<P> a, vector<P> b) { vector<P> res = a; rep(i,0,sz(b)) { res = polygonCut(res, b[i], b[(i + 1) % sz(b)]); } return res; }</pre>	

8.4 Misc. Point Set Problems

ClosestPair.h
Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
<pre>typedef Point<ll> P; pair<P, P> closest(vector<P> v) { assert(sz(v) > 1); set<P> S; sort(all(v), [](P a, P b) { return a.y < b.y; }); pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}}; int j = 0; for (P p : v) { P d(1 + (ll)sqrt(ret.first), 0); while (v[j].y <= p.y - d.x) S.erase(v[j++]); auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d); for (; lo != hi; ++lo) ret = min(ret, {(ll)lo - p).dist2(), {ll}lo, p}); S.insert(p); } return ret.second; }</pre>	

ManhattanMST.h
Description: Given N points, returns up to 4*N edges, which are guaran-teed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = -p.x - q.x + -p.y - q.y$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"Point.h"	df6f59, 23 lines
<pre>typedef Point<int> P;</pre>	

```
vector<array<int, 3>> manhattanMST(vector<P> ps) {
    vi id(sz(ps));
    iota(all(id), 0);
    vector<array<int, 3>> edges;
    rep(k,0,4) {
        sort(all(id), [&](int i, int j) {
            return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
        map<int, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
                int j = it->second;
                P d = ps[i] - ps[j];
                if (d.y > d.x) break;
                edges.push_back({d.y + d.x, i, j});
            }
            sweep[-ps[i].y] = i;
        }
        for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
    }
    return edges;
}
```

ChebyshevDistance.h
Description: Coordinate transformation to convert between Manhattan and Chebyshev distances. Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$ in the original space becomes Chebyshev distance $\max(|u_1 - u_2|, |v_1 - v_2|)$ in the rotated/scaled space where $(u, v) = (x + y, x - y)$. Chebyshev distance $\max(|x_1 - x_2|, |y_1 - y_2|)$ in the original space becomes half of Man-hattan distance $\frac{1}{2}(|u_1 - u_2| + |v_1 - v_2|)$ in the rotated/scaled space where $(u, v) = (x + y, x - y)$. Useful for mapping tilted search regions (Manhattan) to axis-aligned squares (Chebyshev/2D Range Queries).
Time: $\mathcal{O}(1)$

"Point.h"	240470, 9 lines
<pre>template<class T> Point<T> toChebyshev(Point<T> p) { return Point<T>(p.x + p.y, p.x - p.y); } template<class T> Point<double> toManhattan(Point<T> p) { return Point<double>((p.x + p.y) / 2.0, (p.x - p.y) / 2.0); }</pre>	

kdTree.h
Description: KD-tree (2d, can be extended to 3d)

"Point.h"	bac5b0, 63 lines
<pre>typedef long long T; typedef Point<T> P; const T INF = numeric_limits<T>::max(); bool on_x(const P& a, const P& b) { return a.x < b.x; } bool on_y(const P& a, const P& b) { return a.y < b.y; } struct Node { P pt; // if this is a leaf, the single point in it T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds Node *first = 0, *second = 0; T distance(const P& p) { // min squared distance to a point T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x); T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y); return (P(x,y) - p).dist2(); } Node(vector<P>&& vp) : pt(vp[0]) { for (P p : vp) { x0 = min(x0, p.x); x1 = max(x1, p.x);</pre>	

```

    y0 = min(y0, p.y); y1 = max(y1, p.y);
}
if (vp.size() > 1) {
    // split on x if width >= height (not ideal...)
    sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
    // divide by taking half the array for each child (not
    // best performance with many duplicates in the middle)
    int half = sz(vp)/2;
    first = new Node({vp.begin(), vp.begin() + half});
    second = new Node({vp.begin() + half, vp.end()});
}
}
};

struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

    pair<T, P> search(Node *node, const P& p) {
        if (!node->first) {
            // uncomment if we should not find the point itself:
            // if (p == node->pt) return {INF, P()};
            return make_pair((p - node->pt).dist2(), node->pt);
        }

        Node *f = node->first, *s = node->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if (bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }

    // find nearest point to a point, and its squared distance
    // (requires an arbitrary operator< for Point)
    pair<T, P> nearest(const P& p) {
        return search(root, p);
    }
};
```

DelaunayTriangulation.h
Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.

Time: $\mathcal{O}(n^2)$

"Point.h", "3dHull.h" c0e7bc, 10 lines

```

template<class P, class F>
void delaunay(vector<P>& ps, F trifun) {
    if (sz(ps) == 3) { int d = (ps[0].cross(ps[1], ps[2]) < 0);
        trifun(0,1+d,2-d); }
    vector<P3> p3;
    for (P p : ps) p3.emplace_back(p.x, p.y, p.dist2());
    if (sz(ps) > 3) for(auto t:hull3d(p3)) if ((p3[t.b]-p3[t.a]).
        cross(p3[t.c]-p3[t.a]).dot(P3(0,0,1)) < 0)
        trifun(t.a, t.c, t.b);
}
```

FastDelaunay.h
Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ... }, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h" eefd5f, 88 lines

```

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}

Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{0}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}

void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
    for (;;) {
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
    }
}
```

```

        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
    return pts;
}
```

8.5 3D PolyhedronVolume.h
Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```

template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h
Description: Class to handle points in 3D space. T can be e.g. double or long long.

8058ae, 32 lines

```

template<class T> struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(R p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0, pi]
    double theta() const { return atan2(sqrt(x*x+y*y),z); }
    P unit() const { return *this/(T)dist(); } //makes dist()==1
    //returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }
    //returns point rotated 'angle' radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle); P u = axis.unit();
        return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

"Point3D.h"5b45fc, 49 lines

```
typedef Point3D<double> P3;
```

```
struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};
```

```
struct F { P3 q; int a, b, c; };
```

```
vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);
```

```
    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
            int nw = sz(FS);
            rep(j,0,nw) {
                F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
                C(a, b, c); C(a, c, b); C(b, c, a);
            }
            for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
                A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
            return FS;
        };
    }
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius *radius* between the points with azimuthal angles (longitude) *f1* (ϕ_1) and *f2* (ϕ_2) from *x* axis and zenith angles (latitude) *t1* (θ_1) and *t2* (θ_2) from *z* axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. *dx**radius is then the difference between the two points in the *x* direction and *d**radius is the total distance between the points.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
```

```
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Strings (9)

KMP.h

Description: *pi*[*x*] computes the length of the longest prefix of *s* that ends at *x*, other than *s*[0...*x*] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

d4375c, 16 lines

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

KMPAutomaton.h

Description: Builds a deterministic finite automaton (DFA) for KMP string matching. *next*[*i*][*c*] returns the next state after appending character *c* (0-indexed offset from 'a') to a matched prefix of length *i*. If absorbing is true, the automaton stays in state *m* indefinitely once a full match is found (useful for "contains" queries). Here $\Sigma = 26$.

Time: $\mathcal{O}(m\Sigma)$

ac78aa, 17 lines

```
vector<vi> buildAutomaton(int m, const vi& LPS, const string& P
, bool absorbing = false) {
    vector<vi> nxt(m + 1, vi(26));
    for (int matched = 0; matched <= m; matched++) {
        for (int ch = 0; ch < 26; ch++) {
            if (absorbing && matched == m) {
                nxt[matched][ch] = m;
                continue;
            }
            int nj = matched;
            if (nj == m) nj = LPS[m - 1];
            while (nj > 0 && P[nj] - 'a' != ch) nj = LPS[nj - 1];
            if (P[nj] - 'a' == ch) nj++;
            nxt[matched][ch] = nj;
        }
    }
    return nxt;
}
```

Zfunc.h

Description: *z*[*i*] computes the length of the longest common prefix of *s*[*i*] and *s*, except *z*[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i,1,sz(S)) {
```

```
        z[i] = i >= r ? 0 : min(r - i, z[i - 1]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

Manacher.h

Description: For each position in a string, *p*[1][*i*] = (max odd length palindrome centered at *i*) / 2 [floor division], *p*[0][*i*] = same for even, it considers the right center. iterate over *p*[1][*i*] and *p*[0][*i*] to find the longest palindromic substring.

Time: $\mathcal{O}(N)$

78c61e, 20 lines

```
struct Manacher {
    vi p[2];
    Manacher(const string &s) {
        int n = sz(s);
        p[0].resize(n + 1);
        p[1].resize(n);
        rep(z, 0, 2) for(int i = 0, l = 0, r = 0; i < n; i++) {
            int t = r - i + !z;
            if(i < r) p[z][i] = min(t, p[z][l + t]);
            int L = i - p[z][i], R = i + p[z][i] - !z;
            while(L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
                p[z][i]++, L--, R++;
            if (R > r) l = L, r = R;
        }
    }
    bool is_palindrome(int l, int r) { // [l, r] inclusive
        int mid = (l + r + 1) / 2, len = r - l + 1;
        return 2 * p[len % 2][mid] + len % 2 >= len;
    }
};
```

PalindromicTree.h

Description: Palindromic Tree (Eertree). Stores all distinct palindromic substrings. *cnt* contains the number of palindromic suffixes of the node. *oc* tracks occurrences (call *calc*.occurrences after building).

Time: $\mathcal{O}(N)$ build, $\mathcal{O}(N)$ for occurrence count.

a44664, 59 lines

```
struct PalindromicTree {
    struct Node {
        int nxt[26], len, st, en, link, cnt, oc;
    };

```

```
    string s;
    vector<Node> t;
    int sz, last;
```

```
    PalindromicTree(const string& _s = "") : s(_s) {
        int n = sz(s);
        t.assign(n + 3, {});
        sz = last = 2;
        t[1].len = -1; t[1].link = 1;
        t[2].len = 0; t[2].link = 1;
        rep(i, 0, n) extend(i);
    }
```

```
    bool extend(int pos) {
        int cur = last, ch = s[pos] - 'a';
        while (pos - 1 - t[cur].len < 0 || s[pos - 1 - t[cur].len]
            != s[pos]) {
            cur = t[cur].link;
        }
```

```
        if (t[cur].nxt[ch]) {
```

```
        last = t[cur].nxt[ch];
        t[last].oc++;
        return false;
    }

    sz++; last = sz;
    t[sz].len = t[cur].len + 2;
    t[cur].nxt[ch] = sz;
    t[sz].en = pos;
    t[sz].st = pos - t[sz].len + 1;
    t[sz].oc = 1;

    if (t[sz].len == 1) {
        t[sz].link = 2;
        t[sz].cnt = 1;
        return true;
    }

    cur = t[cur].link;
    while (pos - 1 - t[cur].len < 0 || s[pos - 1 - t[cur].len]
        != s[pos]) {
        cur = t[cur].link;
    }

    t[sz].link = t[cur].nxt[ch];
    t[sz].cnt = 1 + t[t[sz].link].cnt;
    return true;
}

void calc_occurrences() {
    for (int i = sz; i >= 3; i--) {
        t[t[i].link].oc += t[i].oc;
    }
}
};
```

GeneralizedPalindromicTree.h
Description: Generalized Palindromic Tree (Eertree) for multiple strings. in_how_many counts how many distinct strings contain each palindromic substring. A string of length L can add a maximum of L new nodes to the Palindromic Tree.
Time: $\mathcal{O}(L)$ to add a string of length L .

```
struct GeneralizedPalindromicTree {
    struct Node {
        int nxt[26], len, st, en, link, cnt, oc;
    };

    string s;
    vector<Node> t;
    int sz, last;
    vector<int> last_seen, in_how_many;

    // Allocate memory exactly for the sum of lengths in the
    // current test case
    GeneralizedPalindromicTree(int max_len) {
        t.assign(max_len + 3, {});
        last_seen.assign(max_len + 3, 0);
        in_how_many.assign(max_len + 3, 0);
        sz = last = 2;
        t[1].len = -1; t[1].link = 1;
        t[2].len = 0; t[2].link = 1;
    }

    bool extend(int pos) {
        int cur = last, ch = s[pos] - 'a';
        while (pos - 1 - t[cur].len < 0 || s[pos - 1 - t[cur].len]
            != s[pos]) {
            cur = t[cur].link;
        }
    }
};
```

```
    }

    if (t[cur].nxt[ch]) {
        last = t[cur].nxt[ch];
        t[last].oc++;
        return false;
    }

    sz++; last = sz;
    t[sz].len = t[cur].len + 2;
    t[cur].nxt[ch] = sz;
    t[sz].en = pos;
    t[sz].st = pos - t[sz].len + 1;
    t[sz].oc = 1;

    if (t[sz].len == 1) {
        t[sz].link = 2;
        t[sz].cnt = 1;
        return true;
    }

    cur = t[cur].link;
    while (pos - 1 - t[cur].len < 0 || s[pos - 1 - t[cur].len]
        != s[pos]) {
        cur = t[cur].link;
    }

    t[sz].link = t[cur].nxt[ch];
    t[sz].cnt = 1 + t[t[sz].link].cnt;
    return true;
}

void add_string(const string& _s, int string_id) {
    s = _s;
    last = 2; // Crucial: Reset to root for the new string
    vector<int> nodes;

    for (int i = 0; i < sz(s); i++) {
        extend(i);
        nodes.push_back(last);
    }

    // Push string presence up the suffix links
    for (int u : nodes) {
        int curr = u;
        // Stop early if this node was already seen in the
        // current string
        while (curr > 2 && last_seen[curr] != string_id) {
            last_seen[curr] = string_id;
            in_how_many[curr]++;
            curr = t[curr].link;
        }
    }
}
};
```

MinRotation.h
Description: Finds the lexicographically smallest rotation of a string.
Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());
Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    rep(b,0,N) rep(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break;}
    }
    return a;
}
```

SuffixArray.h
Description: Builds suffix array for a string. sa[i] is the starting index (0-based) of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n+1$, and sa[0] = n. rank[i] is the rank (1-based) of the i th suffix (0-based) in the initial string. rank[i] is of size $n+1$, and rank[n] = 0. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars. In sa[i] and lcp[i], the index i is relative to sorted suffixes. sa[i] and lcp[i] ask: "Tell me about the suffix that ended up at position i after sorting." In rank[i], the index i is relative to the initial string. rank[i] asks: "Where did the suffix starting at index i in the original string end up after sorting?"
Time: $\mathcal{O}(n \log n)$

```
"RMQ.h" 1c8e02, 32 lines

struct SuffixArray {
    vi sa, lcp, rank;
    // RMQ<int> table;
    int n;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); n = sz(s); int k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                s[i + k] == s[j + k]; k++);
        rank = x;
        // table = RMQ(lcp);
    }

    // int get_lcp(int i, int j) { // lcp of ith and jth
    //     suffixes in the original string
    //     if (i == j) return n - 1 - i;
    //     int u = rank[i], v = rank[j];
    //     if (u > v) swap(u, v);
    //     return table.query(u + 1, v + 1);
    // }
};
```

SuffixTree.h
Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).
Time: $\mathcal{O}(26N)$

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

    void ukkadd(int i, int c) { suff:
        if (r[v]<=q) {
            if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
                p[m++]=v; v=s[v]; q=r[v]; goto suff; }
            v=t[v][c]; q=l[v];
        }
    }
};
```

```
    if (q==-1 || c==toi(a[q])) q++; else {
        l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
        p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
        l[v]=q; p[v]=m; t[p[m]][toi(a[l[m])]]=m;
        v=s[p[m]]; q=l[m];
        while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
        if (q==r[m]) s[m]=v; else s[m]=m+2;
        q=r[v]-(q-r[m]); m+=2; goto suff;
    }
}

SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
}

// example: find longest common substring (uses ALPHA = 28)
pii best;
int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
        mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
        best = max(best, {len, r[node] - len});
    return mask;
}

static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
}
};
```

Hashing.h

Description: Unified Rolling Hash. Supports static range queries via build() and get(l, r), as well as O(1) dynamic sliding window operations (add/remove front/back). Dynamic operations update the 'hash' and 'len' state, independent of the 'h' array.
Time: $\mathcal{O}(N)$ build, $\mathcal{O}(1)$ updates and queries.

```
// H1(n, 131, 1e9+7)
// H2(n, 137, 1e9+9)
struct RollingHash {
    ll BASE, MODH, invBASE, hash, len;
    vl p, h;
    RollingHash(int sz, ll B, ll M) : p(sz + 1), h(sz + 1),
        BASE(B), MODH(M) {
        hash = len = 0;
        p[0] = 1;
        for (int i = 1; i <= sz; i++) {
            p[i] = (p[i - 1] * BASE) % MODH;
        }
        invBASE = modpow(BASE, MODH - 2, MODH);
    }
    static ll modpow(ll a, ll e, ll m) {
        ll r = 1;
        while (e) {
            if (e & 1) r = r * a % m;
            a = a * a % m;
            e >>= 1;
        }
        return r;
    }
};
```

Hashing Hashing2D AhoCorasick

```
void build(const string &s) {
    int n = s.size();
    for (int i = 0; i < n; i++) {
        h[i + 1] = (h[i] * BASE + s[i]) % MODH;
    }
    hash = h[n];
    len = n;
}

ll get(int l, int r) {
    ll res = (h[r + 1] - h[l] * p[r - l + 1]) % MODH;
    return (res + MODH) % MODH;
}

// Dynamic sliding window operations
void add_back(char c) {
    hash = (hash * BASE + c) % MODH;
    len++;
}

void add_front(char c) {
    hash = (hash + c * p[len] % MODH) % MODH;
    len++;
}

void remove_back(char c) {
    hash = (hash - c + MODH) % MODH;
    hash = (hash * invBASE) % MODH;
    len--;
}

void remove_front(char c) {
    hash = (hash - c * p[len - 1] % MODH + MODH) % MODH;
    len--;
}
};
```

Hashing2D.h

Description: 2D Rolling Hash for grid pattern matching. Utilizes two bases to maintain spatial integrity across rows and columns. Supports O(1) sub-grid hash queries via 2D prefix structures.
Time: $\mathcal{O}(R \times C)$ build, $\mathcal{O}(1)$ query.

```
// RollingHash2D H1(n, m, 313, 317, 1000000007)
// RollingHash2D H2(n, m, 337, 347, 1000000009)
struct RollingHash2D {
    ll BR, BC, MODH;
    vector<vector<ll>> h;
    vector<ll> pR, pC;

    RollingHash2D(int szR, int szC, ll BaseR, ll BaseC, ll M)
        : pR(szR + 1), pC(szC + 1), h(szR + 1, vector<ll>(szC + 1, 0)),
        BR(BaseR), BC(BaseC), MODH(M) {

        pR[0] = 1;
        pC[0] = 1;
        rep(i, 1, szR + 1) pR[i] = (pR[i - 1] * BR) % MODH;
        rep(i, 1, szC + 1) pC[i] = (pC[i - 1] * BC) % MODH;
    }

    void build(const vector<string> &grid) {
        int R = sz(grid);
        int C = grid.empty() ? 0 : sz(grid[0]);

        rep(i, 1, R + 1) {
            rep(j, 1, C + 1) {
                ll val = grid[i - 1][j - 1];

                // 2D Hash Prefix Formula:
                // h[i][j] = h[i-1][j]*BR + h[i][j-1]*BC - h[i-1][j-1]*BR*BC + val
                ll t1 = (h[i - 1][j] * BR) % MODH;
                ll t2 = (h[i][j - 1] * BC) % MODH;
```

```
                ll t3 = (h[i - 1][j - 1] * BR) % MODH * BC % MODH;

                h[i][j] = (t1 + t2 - t3 + val) % MODH;
                if (h[i][j] < 0) h[i][j] += MODH;
            }
        }

        // Queries the hash of the subgrid from (r1, c1) to (r2, c2) inclusive (0-indexed)
        ll get(int r1, int c1, int r2, int c2) {
            r1++; c1++; r2++; c2++;

            ll term1 = h[r2][c2];
            ll term2 = h[r1 - 1][c2] * pR[r2 - r1 + 1] % MODH;
            ll term3 = h[r2][c1 - 1] * pC[c2 - c1 + 1] % MODH;
            ll term4 = h[r1 - 1][c1 - 1] * pR[r2 - r1 + 1] % MODH * pC[c2 - c1 + 1] % MODH;

            ll res = term1 - term2 - term3 + term4;
            return (res % MODH + MODH) % MODH;
        }
    };
};
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

```
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vl backp;
    void insert(string& s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int& m = N[n].next[c - first];
            if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if (N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1) {
        rep(i,0,sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
        N.emplace_back(0);

        queue<int> q;
        for (q.push(0); !q.empty(); q.pop()) {
            int n = q.front(), prev = N[n].back;
            rep(i,0,alpha) {
                int &ed = N[n].next[i], y = N[prev].next[i];
                if (ed == -1) ed = y;
```

```
        else {
            N[ed].back = y;
            (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                = N[y].end;
            N[ed].nmatches += N[y].nmatches;
            q.push(ed);
        }
    }
}

vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}

vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}
```

```
void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).

Time: $\mathcal{O}(N \log N)$

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});

Time: $\mathcal{O}(k \log \frac{n}{k})$

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to; p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}

template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in [a, b] that maximizes f(i), assuming that f(a) < ... < f(i) ≥ ... ≥ f(b). To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).

Usage: int ind = ternSearch(0, n-1, [&](int i){return a[i];});

Time: $\mathcal{O}(\log(b - a))$

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
```

```
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    rep(i, a+1, b+1) if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

CoordinateCompression.h

Description: Coordinate Compression helper. 'get(x)' returns the rank of value 'x' (0-indexed). 'c[i]' returns the original value with rank 'i'.

Time: Construction $\mathcal{O}(N \log N)$, Query $\mathcal{O}(\log N)$.

```
struct Compress {
    vector<ll> vals;
    Compress(vector<ll> &v) : vals(v) {
        sort(all(vals));
        vals.erase(unique(all(vals)), vals.end());
    }
    int get(ll x) {
        return lower_bound(all(vals), x) - vals.begin();
    }
    ll operator[](int i) { return vals[i]; }
    int size() { return vals.size(); }
};
```

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i, 0, sz(S)) {
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p[S[i], 0]);
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S ≤ t such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

```
int knapsack(vi w, int t) {
    int a = 0, b = 0, x;
    while (b < sz(w) && a + w[b] <= t) a += w[b++];
    if (b == sz(w)) return a;
    int m = *max_element(all(w));
    vi u, v(2*m, -1);
    v[a+m-t] = b;
    rep(i, b, sz(w)) {
        u = v;
        rep(x, 0, m) v[x+w[i]] = max(v[x+w[i]], u[x]);
        for (x = 2*m; --x > m;) rep(j, max(0, u[x]), v[x])
            v[x-w[j]] = max(v[x-w[j]], j);
    }
    for (a = t; v[a+m-t] < 0; a--);
    return a;
}
```

PermutationsHashing.h

Description: N-dimensional Zobrist/Multiset Hashing struct. Assigns a unique N-tuple of random integers to elements to probabilistically verify multiset equality, permutation matching, or subarray compositions. Supports commutative merging via addition, removal via subtraction, and parity tracking via XOR.

116d07, 33 lines

```
mt19937_64 rng(chrono::system_clock::now().time_since_epoch().count());
ll rand(ll l, ll r) return uniform_int_distribution<ll>(l, r)(rng);

template <size_t N = 5>
struct PermHash {
    array<ll, N> h;
    PermHash() { h.fill(0); }
    static PermHash random() {
        PermHash res;
        for (size_t i = 0; i < N; ++i) res.h[i] = rand(1, 1e9);
        // Replace with your RNG
        return res;
    }
    PermHash operator+(const PermHash& o) const {
        PermHash res;
        for (size_t i = 0; i < N; ++i) res.h[i] = h[i] + o.h[i];
        return res;
    }
    PermHash operator-(const PermHash& o) const {
        PermHash res;
        for (size_t i = 0; i < N; ++i) res.h[i] = h[i] - o.h[i];
        return res;
    }
    PermHash operator^(const PermHash& o) const {
        PermHash res;
        for (size_t i = 0; i < N; ++i) res.h[i] = h[i] ^ o.h[i];
        return res;
    }
    bool isEmpty() const {
        for (size_t i = 0; i < N; ++i) if (h[i]) return false;
        return true;
    }
    bool operator==(const PermHash& o) const { return h == o.h; }
};
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$.

Time: $\mathcal{O}((N + (hi - lo)) \log N)$

d38d2b, 18 lines

```
struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
```

```
void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
        best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
}

void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
```

SosDP.h

Description: SOS (Sum Over Subsets) DP / Fast Zeta Transform. Forward Pass (Zeta): Build sums from original (use +=). Backward Pass (Mobius): Invert to extract original (use -=).

b05426, 19 lines

```
// Cheat Sheet (inside the if condition):
// Sum over Subsets (0 flows to 1):
// dp[bit_on] += dp[bit_off]; // Subsets to Superset
// Undo Sum over Subsets:
// dp[bit_on] -= dp[bit_off];
// Sum over Supersets (1 flows to 0):
// dp[bit_off] += dp[bit_on]; // Supersets to Subset
// Undo Sum over Supersets:
// dp[bit_off] -= dp[bit_on];

void sos(vector<ll>& dp, int LOG) {
    rep(i, 0, LOG) rep(mask, 0, 1<<LOG) {
        if (mask & (1 << i)) {
            int bit_on = mask;
            int bit_off = mask ^ (1 << i);
            // dp[bit_on] += dp[bit_off];
        }
    }
}
```

10.4 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); }); converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- x & -x is the least bit in x.
- **for** (**int** x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- c = x&-x, r = x+c; ((r^x) >> 2)/c) | r is the next number after x with the same number of bits set.

- rep(b, 0, K) rep(i, 0, (1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)]; computes all sums of subsets.

10.5.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- #pragma GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- #pragma GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a \% b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: ./a.out < input.txt
Time: About 5x as fast as cin/scanf.

7b3c70, 17 lines

```
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

745db2, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20]; // Reserve 450 MB of memory
    globally
void* operator new(size_t s) {
    static size_t i = sizeof buf;
    assert (s < i);
    return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h"2dd6c9, 10 lines

```
template<class T> struct ptr {
    unsigned ind;
    ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
        assert(ind < sizeof buf);
    }
    T& operator*() const { return *(T*)(buf + ind); }
    T* operator->() const { return &*this; }
    T& operator[](int a) const { return (&*this)[a]; }
    explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.

Usage: vector<vector<int, small<int>>> ed(N);bb66d4, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
    typedef T value_type;
    small() {}
    template<class U> small(const U&) {}
    T* allocate(size_t n) {
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T*)(buf + buf_ind);
    }
    void deallocate(T*, size_t) {}
};
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern "_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)". Not all are described here; grep for _mm_ in /usr/lib/gcc/*4.9/include/ for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and #define _SSE_ and _MMX_ before including it. For aligned memory use _mm_malloc(size, 32) or int buf[N] alignas(32), but prefer loadu/storeu.

551b82, 43 lines

```
#pragma GCC target ("avx2") // or sse4.1
#include "emmintrin.h"

typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi*)&(x))

// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm

// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
    int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
mi zero() { return _mm256_setzero_si256(); }
```

```
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short* a, short* b) {
    int i = 0; ll r = 0;
    mi zero = _mm256_setzero_si256(), acc = zero;
    while (i + 16 <= n) {
        mi va = L(a[i]), vb = L(b[i]); i += 16;
        va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
        mi vp = _mm256_madd_epi16(va, vb);
        acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
            _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)));
    }
    union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[i];
    for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <- equiv
    return r;
}
```

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree